A PROJECT-B REPORT

On

TraffiSense: An Explainable, Digital-Twin-Driven Intelligent Traffic Management and Violation Detection System

Submitted in partial fulfillment of the requirement of

University of Mumbai for the Degree of

Bachelor of Technology

In

Computer Engineering

Submitted By

Sachin Nair

Ranjith Iyer

Aditya Vijay Kumar

Supervisor

Dr. Siddharth Hariharan

Department Of Computer Engineering

PILLAI COLLEGE OF ENGINEERING(AUTONOMOUS)

New Panvel – 410 206

UNIVERSITY OF MUMBAI

Academic Year 2026 - 27

DEPARTMENT OF COMPUTER ENGINEERING

Pillai College of Engineering

New Panvel – 410 206

CERTIFICATE

This is to certify that the requirements for the project report entitled '**TraffiSense: An Explainable, Digital-Twin-Driven Intelligent Traffic Management and Violation Detection System**' have been successfully completed by the following students:

Name	**Roll No.**
Sachin Nair	B737
Ranjith Iyer	B718
Aditya Vijay Kumar	B702

In partial fulfillment of Bachelor of Technology, University of Mumbai in the Department of Computer Engineering, Pillai College of Engineering, New Panvel – 410 206 during the Academic Year 2026 – 2027.

_____

Dr. Siddharth Hariharan
Supervisor

_____

Dr. Sharvari Govilkar
Head, Dept. of Computer Engineering

_____

Dr. Sandeep M. Joshi
Principal

DEPARTMENT OF COMPUTER ENGINEERING

Pillai College of Engineering

New Panvel – 410 206

PROJECT APPROVAL FOR B.Tech

This project entitled "**TraffiSense: An Explainable, Digital-Twin-Driven Intelligent Traffic Management and Violation Detection System**" by Sachin Nair, Ranjith Iyer and Aditya Vijay Kumar is approved for the degree of Bachelor of Technology in Computer Engineering.

Examiners:

1. _____

2. _____

Supervisors:

1. _____

2. _____

Chairman:

1. _____

Date:

Place:

DEPARTMENT OF COMPUTER ENGINEERING
Pillai College of Engineering
New Panvel – 410 206

DECLARATION

We declare that this written submission for the B.Tech project entitled "**TraffiSense: An Explainable, Digital-Twin-Driven Intelligent Traffic Management and Violation Detection System**" represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any ideas / data / fact / source in our submission. We understand that any violation of the above will cause disciplinary action by the institute and also evoke penal action from the sources which have not been properly cited or from whom prior permission have not been taken when needed.

Project Group Members:

Sachin Nair & Sign: _____

Ranjith Iyer & Sign: _____

Aditya Vijay Kumar & Sign: _____

Date:

Place:

Table of Contents

Abstract i

List of Figures ii

List of Tables iii

1. Introduction 1

    1.1 Fundamentals 1

    1.2 Objectives 4

    1.3 Scope 4

    1.4 Organization of the Project Report 6

2. Literature Survey 7

    2.1 Introduction 7

    2.2 Literature Review 8

        2.2.1 Adaptive Traffic Signal Control 8

        2.2.2 Emergency-Vehicle Priority 8

        2.2.3 Real-Time Object Detection 8

        2.2.4 Indian Traffic Datasets 9

        2.2.5 Helmet Violation Detection 9

        2.2.6 Triple-Riding Detection 9

        2.2.7 Automated E-Ticket Generation 10

        2.2.8 Low-Resolution Number-Plate Recognition 10

    2.3 Summary of Literature Survey 11

3. Implemented System: TraffiSense 14

3.1 Overview 14

3.1.1 Existing System Architecture 16

3.1.2 Proposed System Architecture 16

3.2 Implementation Details 18

3.2.1 Algorithm / Methodology / Techniques 20

3.2.2 Use Case Diagram / Activity Diagram / DFD 28

3.2.3 Hardware and Software Specifications 34

4. Result and Discussion 38

4.1 Sample of Inputs / Dataset / Database Used / Outputs / Screen Shots 38

4.1.1 Sample Inputs 38

4.1.2 Dataset Used 39

4.1.3 Database Used 39

4.1.4 Outputs 41

4.1.5 Screen Shots 41

4.2 Evaluation Parameters 49

4.2.1 Detection Metrics 50

4.2.2 Decision Metrics 50

4.2.3 Measurement Metrics 51

4.2.4 System and Functional Metrics 51

4.2.5 Evaluation Protocol 51

4.3 Performance Evaluation 52

4.3.1 Vehicle Detection 52

4.3.2 Helmet Check 54

4.3.3 Triple-Riding Check 55

4.3.4 Signal State and Red-Light Check 56

4.3.5 Emergency-Vehicle Exemption 57

4.3.6 Number-Plate Reading 57

4.3.7 Speed Estimation 58

4.3.8 Platform-Level Results and Functional Testing 59

4.3.9 Adaptive Signal Control in Simulation 60

4.3.10 Comparison with the Project-A Proposal 64

4.3.11 Discussion 65

5. Summary 67

References 70

Acknowledgement 71

Abstract

This project presents TraffiSense: An Explainable, Digital-Twin-Driven Intelligent Traffic Management and Violation Detection System, an evidence-first traffic enforcement platform that turns recorded traffic video into explainable violation decisions for Indian roads. The core problem addressed is that two-wheeler offences such as riding without a helmet and triple riding, and signal offences such as red-light jumping, are still enforced mostly by officers watching junctions or control-room screens, which is slow, tiring and prone to single-glance mistakes. TraffiSense integrates several deep-learning components into one web application: a YOLO11 vehicle detector fine-tuned on the Indian Driving Dataset with ByteTrack multi-object tracking; a rider-level helmet classifier; a rider-count classifier for triple riding; a signal-state model with temporal smoothing and automatic stop-line estimation for red-light jumping; a calibrated speed estimator that reports a 90 % speed range; an Indian number-plate reader that prints a plate only when it is read with certainty; and a CLIP-based probe that exempts ambulances, police vehicles and fire engines. Every decision is taken over many frames of the same tracked vehicle, carries a plain-language explanation, a confidence value and fingerprinted evidence frames and clips, and is routed to an automatic e-challan only when the model is certain. Uncertain cases go to a human review queue, and repeat detections of the same vehicle are merged so that a vehicle never receives two challans. The platform is built with a React and TypeScript frontend, a FastAPI backend and an SQLite database, and includes secure sign-up with verification, a digital-twin view, lane estimation and analytics. For signal control, a Unity simulator of an Indian four-arm junction is used to evaluate PRAVAH, a person-weighted, red-patience-aware adaptive signal controller developed in this project; in simulation it gave the lowest mean wait per person of the four controllers compared under every one of twelve tested sets of simulator assumptions, and the platform can replay the traffic measured in an uploaded video through it. TraffiSense is a decision-support prototype: it favours “not sure” over a wrong challan.

List of Figures

Fig 1.1 High-level view of the TraffiSense enforcement workflow 3
Fig 2.1 Families of AI techniques reviewed in the literature 7
Fig 3.1 Existing enforcement flow and its limitations 16
Fig 3.2 Proposed architecture of TraffiSense 17
Fig 3.3 Runtime and deployment view of TraffiSense 18
Fig 3.4 Map of the 18 frontend pages grouped by purpose 19
Fig 3.5 Video analysis pipeline of TraffiSense 21
Fig 3.6 Routing of a finding to challan, review or exemption 25
Fig 3.7 One case per vehicle: merging repeat detections 26
Fig 3.8 Use case diagram of the TraffiSense platform 29
Fig 3.9 Level-0 data flow diagram 30
Fig 3.10 Level-1 data flow diagram 31
Fig 3.11 Activity diagram of the video analysis workflow 33
Fig 3.12 Sequence diagram of a video analysis 34
Fig 4.1 Sign-in and account creation pages 42
Fig 4.2 Dashboard with video guidance and upload 42
Fig 4.3 Video analyses 43
Fig 4.4 Analysis result with video, overlays and findings 43
Fig 4.5 Violations with decision path 44
Fig 4.6 Human review of an uncertain case 44
Fig 4.7 Issued challans 45
Fig 4.8 Challan detail with evidence 45
Fig 4.9 First page of a generated e-challan PDF 46
Fig 4.10 Evidence gallery 47
Fig 4.11 Digital twin of the scene 47
Fig 4.12 Lane detection from vehicle movement 48
Fig 4.13 Analytics 48
Fig 4.14 Models and system 49
Fig 4.15 Settings 49
Fig 4.16 Validation precision, recall and mAP of the vehicle detector during training 53
Fig 4.17 Per-class precision, recall and mAP@0.5 on the IDD test split 54
Fig 4.18 Helmet check on seven unseen videos: first version and final rule 55
Fig 4.19 Emergency vehicles caught against ordinary vehicles wrongly flagged 57
Fig 4.20 Routing of raw findings by check across all analyses 59
Fig 4.21 Processing time against video length 60
Fig 4.22 The Unity junction simulator: an Indian four-arm junction with mixed traffic 62
Fig 4.23 Mean wait per person by controller and scenario (lower is better) 63

List of Tables

Table 1.1 Key concepts used in TraffiSense 3
Table 2.1 Comparison of the technique families 11
Table 2.2 Literature survey summary 12
Table 2.3 Research gaps and the corresponding TraffiSense design decisions 13
Table 3.1 Summary of the implemented TraffiSense system 15
Table 3.2 Limitations of the existing system and the response in TraffiSense 17
Table 3.3 Components of the proposed architecture and their responsibilities 19
Table 3.4 Frontend pages of TraffiSense 20
Table 3.5 Mapping of actors to use cases 30
Table 3.6 Processes and data stores of the level-1 DFD 32
Table 3.7 Main API endpoints of TraffiSense 35
Table 3.8 Hardware specification 35
Table 3.9 Software specification 36
Table 3.10 Configuration parameters 36
Table 4.1 Guidance for input videos shown on the dashboard 38
Table 4.2 Options of an analysis 39
Table 4.3 Datasets used for model development and testing 40
Table 4.4 Tables of the TraffiSense database 40
Table 4.5 Outputs of an analysis 41
Table 4.6 Performance of the vehicle detector on the IDD test split 53
Table 4.7 Performance of the helmet check 55
Table 4.8 Triple-riding check on held-out cameras (980 bikes, 125 triples) 56
Table 4.9 Performance of the signal-state classifier 56
Table 4.10 Emergency-vehicle recognition at different thresholds 57
Table 4.11 Performance of the number-plate reader 58
Table 4.12 Calibrated speed estimation on reference scenes 58
Table 4.13 Processing results of the uploaded videos 59
Table 4.14 Functional test scenarios and observed behaviour 61
Table 4.15 Mean wait per person (s) in simulation: 30 min, 3 seeds, identical arrivals 62
Table 4.16 Robustness of the mean wait per person to the simulator assumptions 64
Table 4.17 The project's rule-based signal controller run through the Python bridge (simulated) 64
Table 4.18 Project-A proposal compared with the Project-B implementation 65
Table 5.1 Components delivered in the integrated prototype 68

Chapter 1

Introduction

1.1 Fundamentals

Two-wheelers form the largest share of road traffic in Indian cities, and they are involved in a large share of road deaths. The Motor Vehicles (Amendment) Act, 2019 makes riding without a protective headgear, carrying more than one pillion rider and jumping a red signal punishable offences. In practice these rules are enforced mainly by traffic personnel who watch a junction, or by operators who watch CCTV screens in a control room. The approach does not scale to the number of cameras that cities now install, it depends on the attention of individual officers, and a decision taken from one glance at one frame is easily wrong when the view is blurred, partly hidden or badly lit.

Deep learning makes it possible for a computer to watch the same footage. Real-time object detectors of the YOLO family locate every vehicle, rider and person in a frame in a single pass [4], and multi-object trackers such as ByteTrack link these detections over time so that each road user keeps a stable identity. Once a motorcycle is tracked, its riders, their heads and its number plate can be examined in every frame in which they are visible, and evidence from many frames can be combined before any decision is taken. Earlier Indian work on two-wheeler enforcement has shown that such pipelines can reach high detection accuracy, but also that low light, occlusion and camera placement still cause errors when a decision rests on a single frame [7].

Accuracy alone is not enough for enforcement. A challan affects a citizen, so the reason for it must be visible, the evidence must be preserved, and a person must be able to check a doubtful case. A wrong challan on an unclear rider or a guessed plate is worse than no challan, and plates on wide camera views are often too small to read at all [10]. A practical enforcement system therefore needs three things at once: reliable multi-frame perception, an explanation and evidence trail for every decision, and a clear rule for when a human must decide.

TraffiSense was proposed in the preceding Project-A as an architecture for explainable, multi-frame violation detection with human review, a digital twin and adaptive signal control. In this Project-B it has been implemented as a working web platform together with a junction

simulator for signal control. An operator uploads recorded traffic video; the platform detects and tracks every vehicle, runs the selected violation checks, reads number plates, collects evidence and decides for every finding whether it becomes an automatic e-challan, goes to a human reviewer, or is exempt because the vehicle is an ambulance, a police vehicle or a fire engine.

TraffiSense is deliberately positioned as an **evidence-first decision-support** system. Its operating rule is precision first: when a model is not sure, the case is sent to a person instead of producing a challan, and a number plate is printed only when it has been read with certainty across frames. Every output is information intended for an authorised human, who remains responsible for enforcement.

Key concepts used in this report

Table 1.1 summarises the technical terms that recur throughout the report. The distinction between a *finding*, a *case* and a *challan* is particularly important, because a single vehicle can produce several findings but must never receive more than one challan.

Figure 1.1 gives a high-level view of how these elements fit together. The operator works through the React web application. The FastAPI server stores the upload and hands it to a background analysis worker, which runs the trained detectors and turns their output into findings with evidence. Each finding is then routed to an automatic challan, to human review, or to exemption.

A further design goal is to make the result easy to trust. For every finding the operator can see the evidence frames before, at and after the moment of the offence, a short clip of the original video, the model's confidence, the reasons it gave in plain language and, where available, the number-plate crop. The same information is printed on the e-challan PDF together with the evidence fingerprint.

Table 1.1: Key concepts used in TraffiSense

Concept	Meaning in the context of TraffiSense
Analysis	One uploaded video together with the checks selected for it, its processing status and results.
Finding	One detection of a possible offence by one detector for one tracked vehicle: time, confidence, model decision and reasons.
Route	Where a finding goes: *confirmed* (automatic challan), *review* (human queue) or *exempt* (emergency vehicle).
Multi-frame decision	A decision combined over all frames in which the same tracked vehicle or rider is visible, never from one frame.
Operating point	The thresholds of a model's decision rule, chosen on held-out cameras so that wrong challans are avoided.
Not sure	The model's answer when the evidence lies between its thresholds; such cases are reviewed by a person.
Plate decision	READ (certain, may be printed), NOT SURE (shown to the reviewer to confirm) or UNREADABLE.
Evidence fingerprint	SHA-256 hash over every evidence file of a finding, printed on the challan.
Duplicate merging	Grouping all findings of one vehicle into one case, so that one vehicle receives at most one challan.
Context only	Output such as lane structure that supports understanding but never creates a challan.
Human-in-the-loop	Design principle that an authorised person decides every case the model is not certain about.

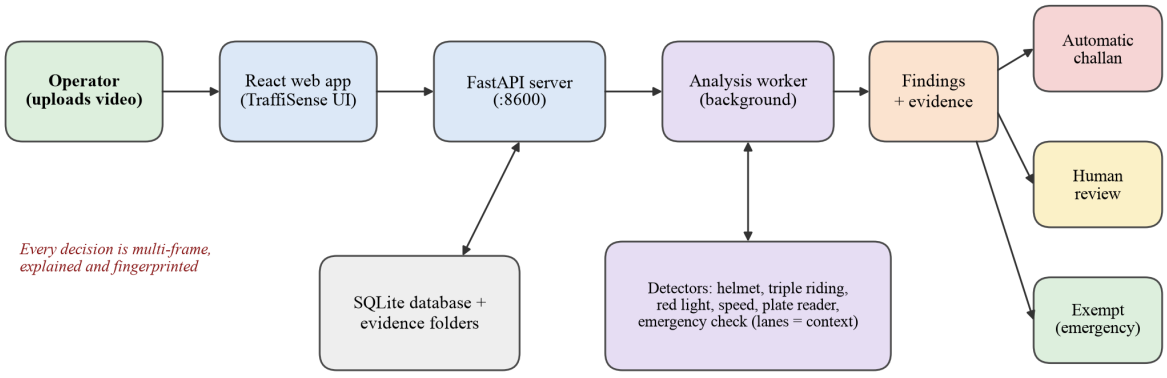

Fig 1.1: High-level view of the TraffiSense enforcement workflow

1.2 Objectives

The objective of this work is to design, implement and demonstrate an explainable, evidence-first traffic enforcement platform for recorded Indian traffic video. The specific objectives are as follows:

1. To detect and track every vehicle in an uploaded video with a detector trained on Indian road scenes and a multi-object tracker, running the detection pass once and sharing it with all checks.
2. To detect riding without a helmet, triple riding, red-light jumping and over-speeding with multi-frame decision rules whose thresholds favour “not sure” over a wrong challan.
3. To read Indian number plates across frames and print a plate only when it is read with certainty.
4. To exempt emergency vehicles (ambulance, police, fire) from challans and record the reason.
5. To attach evidence frames, clips, plain-language reasons and a SHA-256 fingerprint to every finding, and to generate a downloadable e-challan PDF.
6. To route every uncertain finding to a human review queue and guarantee at most one challan per vehicle.
7. To deliver the platform as a secure multi-page web application with verified accounts, analytics, a digital-twin view and lane estimation.
8. To design an adaptive signal controller for Indian mixed traffic and evaluate it against fixed-time and adaptive baselines in a simulator of an Indian junction, clearly labelling every such result as simulated.

1.3 Scope

The scope of the project covers an integrated academic prototype consisting of a React and TypeScript frontend of 18 pages, a FastAPI backend with an SQLite database, and a model layer of trained tools for vehicle detection and tracking, helmet use, triple riding, signal state and red-light crossing, speed estimation, number-plate reading, emergency-vehicle recognition and lane estimation, together with a Unity junction simulator and the PRAVAH signal controller. The prototype supports account creation with verification codes, sign-in and password reset, video upload with a choice of checks, live progress, results with evidence, a human review queue, automatic and reviewed e-challans with PDF download, an evidence gallery, a digital-twin scene, lane detection, analytics, settings, and a simulated signal-timing comparison for the traffic of an analysed video.

The signal-control part of the scope covers a microscopic simulator of an Indian four-arm junction with mixed traffic, red-light jumping, free left turns and bus stops; the PRAVAH controller and three baseline controllers (fixed 30 s, Webster and queue-pressure adaptive); a Python bridge through which controllers written in Python can run the simulated signal; and a page of the platform that replays the traffic measured in an analysed video through the simulator. All signal-timing results are simulated and are labelled as such wherever they appear.

The following boundaries apply. They are part of the project definition:

- TraffiSense analyses **recorded** video uploaded by an operator. It does not connect to live camera streams and does not control real traffic signals; adaptive signal control is implemented and evaluated only in simulation, in a Unity junction simulator that takes the place of the SUMO digital twin proposed in Project-A.
- TraffiSense is a decision-support prototype. Its challans are records produced by the platform for review and demonstration; they are not legally issued notices and are not connected to any government e-challan system.
- Speed is checked only when a one-time camera calibration and a speed limit are supplied, and lane position never creates a challan.
- Verification codes are delivered by e-mail when SMTP settings are provided; without them the platform runs in a clearly marked developer mode.

1.4 Organization of the Project Report

The report is organized as follows:

- **Chapter 1** introduces the fundamentals of camera-based traffic enforcement and adaptive signal control, and states the objectives and the scope of the project.
- **Chapter 2** reviews ten recent works on adaptive signal control, emergency-vehicle priority, real-time object detection, Indian traffic datasets, two-wheeler violation detection and low-resolution number-plate recognition, and summarises the research gaps.
- **Chapter 3** presents the implemented system: the overview, the existing and proposed architectures, the algorithms and techniques including the PRAVAH signal controller and the junction simulator, the use case, data flow, activity and sequence diagrams, and the hardware and software specifications.
- **Chapter 4** describes the inputs, datasets, database and outputs, shows the screens of the platform, defines the evaluation parameters and reports the performance of every component, including signal control in simulation.
- **Chapter 5** summarises the work and the components delivered, and outlines the future scope.
- The references, the list of publications with the survey paper, and the acknowledgement follow.

Chapter 2

Literature Survey

2.1 Introduction

In this chapter the relevant literature is reviewed. It describes the techniques used in camera-based traffic enforcement and adaptive signal control, and the limitations that motivate the design of TraffiSense. Ten recent works (2025–2026) are reviewed, and their summary is presented at the end of the chapter.

Traffic management has moved from officers at the junction and fixed signal plans towards camera-based systems driven by deep learning. Signal timing can now follow measured demand [1] and even be decided by large language models [2], and emergency vehicles can be detected and given priority [3]. Real-time detectors understand traffic scenes from ordinary cameras [4], datasets recorded on Indian roads teach them local vehicle types [5], and dedicated systems judge helmet use [6], count riders [7][8], generate e-tickets [9] and read number plates at low resolution [10].

Figure 2.1 groups the reviewed works into six families: adaptive signal control, emergency-vehicle priority, real-time object detection, Indian traffic datasets, two-wheeler violation detection, and number-plate recognition. The following sections review each work.

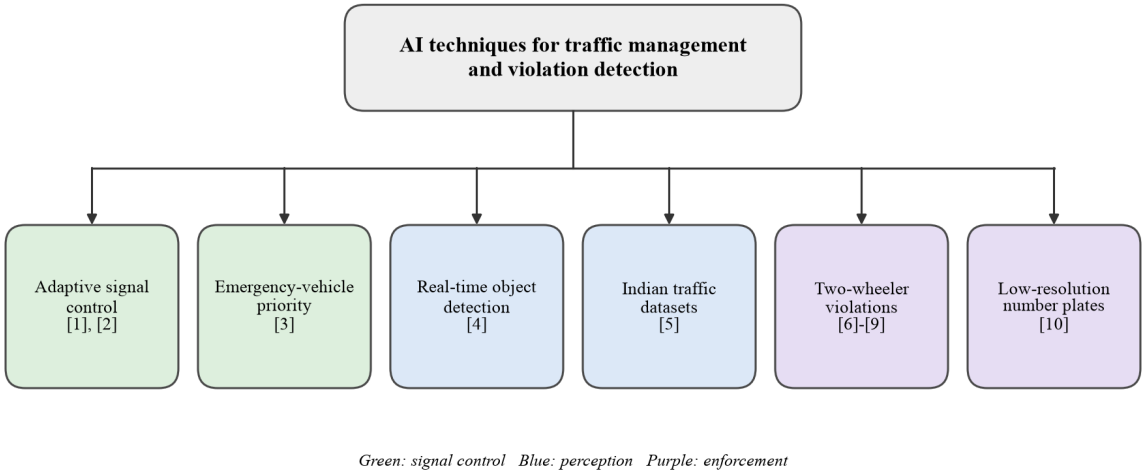

Fig 2.1: Families of AI techniques reviewed in the literature

2.2 Literature Review

Each subsection describes the reviewed work, its method and results, and its main advantages and disadvantages for an Indian enforcement and signal-control system.

2.2.1 Adaptive Traffic Signal Control

Tolani et al. proposed a machine-learning based system for adaptive traffic prediction and control. Proximity sensors placed upstream of the signal measure the speed and density of approaching vehicles, and a model built on the Edge Impulse platform predicts vehicle density and arrival time, so that the signal timing is adjusted in real time instead of following preset timers [1]. Lai et al. introduced LLMLight, in which a large language model acts as the decision agent for signal control: it receives a prompt describing the current traffic and reasons about which phase to serve. A specialised backbone, LightGPT, was trained for the task, and experiments on ten real and synthetic datasets, together with an evaluation by fifteen human experts, showed better generalisation and interpretability than reinforcement-learning controllers [2].

- **Advantages:** green times follow real demand; the language-model agent can explain its choice of phase.
- **Disadvantages:** demand is counted in vehicles, not people; long red waits that provoke red-light jumping are not treated as a risk; both assume lane-based queues.

2.2.2 Emergency-Vehicle Priority

Alruwaili et al. addressed emergency response times with a system that detects ambulances from both sound and video. Sirens are recognised by an LSTM network on MFCC features and ambulances in images by a transfer-learned ResNet18 network, and the result drives Raspberry Pi-based signal prioritisation so that the ambulance's approach receives green [3].

- **Advantages:** two independent cues (sound and image) for detection; low-cost hardware.
- **Disadvantages:** priority is granted as soon as the vehicle is detected, not timed to the queue in front of it; emergency vehicles are not exempted from violation checks.

2.2.3 Real-Time Object Detection

Tian et al. presented YOLOv12, an attention-centric real-time object detector that brings the accuracy of attention mechanisms into the YOLO family while keeping the speed of CNN-based detectors. Its smallest model reaches 40.6 % mAP on COCO with a latency of 1.64 ms on a T4 GPU, outperforming earlier real-time detectors [4].

- **Advantages:** real-time speed with one model for all road users; strong accuracy for its size.
- **Disadvantages:** trained on generic data that lacks Indian vehicle types; a single frame gives no identity over time.

2.2.4 Indian Traffic Datasets

Sharma et al. released the Urban Vision Hackathon dataset, UVH-26, containing 26,646 full-HD images from about 2,800 Bengaluru Safe-City CCTV cameras. The images were annotated by 565 students with 1.8 million bounding boxes over 14 Indian vehicle classes, such as two-wheeler, autorickshaw and tempo-traveller, and detection models trained on the data were released with it [5].

- **Advantages:** elevated CCTV viewpoint like that of junction cameras; local vehicle classes.
- **Disadvantages:** a single city; a very large download.

2.2.5 Helmet Violation Detection

Deshpande et al. developed a two-stage helmet violation detection system for Indian smart-city scenarios, in which a DetectNet_v2 model from the NVIDIA TAO toolkit localises riders and YOLOv8 detects helmet violations and number plates. On 1,715 images from Indian roads it reported 98.56 % helmet detection and 97.6 % number-plate detection accuracy [6].

- **Advantages:** high accuracy on curated images; links the violation directly to the plate.
- **Disadvantages:** decisions on single images; no explicit “not sure” outcome for unclear riders.

2.2.6 Triple-Riding Detection

Deshpande et al. also proposed a triple-riding detection system for surveillance cameras mounted at different heights and angles. A ResNet18-based DetectNet_v2 handles occlusion and distant vehicles, and YOLOv8 counts riders and reads plates, giving 91.42 % triple-rider, 98 % two-wheeler and 81 % number-plate detection accuracy [7]. Saravanan and Rajini classified double and triple riders using a modified Mask R-CNN framework with DenseNet-121 and ResNet-50 networks, reporting 97.93 % triple-rider detection, 97.41 % classification and 98.22 % segmentation accuracy across different weather conditions [8].

- **Advantages:** high accuracy; robustness to camera angle and weather.
- **Disadvantages:** riders of neighbouring bikes can be counted as passengers; plate reading is much weaker than rider counting; single-frame decisions.

2.2.7 Automated E-Ticket Generation

Rawat et al. built DashCop, an end-to-end system that processes dashcam video to detect triple riding and helmet non-compliance and generates e-tickets automatically. A segmentation and cross-association module links riders to their motorcycles, and the annotated RideSafe-400 video dataset was released for evaluation [9].

- **Advantages:** works on video end to end, from detection to the e-ticket; rider–motorcycle association.
- **Disadvantages:** moving dashcam viewpoint; tickets are issued without a human-review step or an evidence fingerprint.

2.2.8 Low-Resolution Number-Plate Recognition

Laroca et al. organised the ICPR 2026 competition on low-resolution license plate recognition using the LRLPR-26 dataset, which contains 20,000 training and 3,000 test tracks, each pairing low- and high-resolution images of the same plate captured by real surveillance cameras. It shows that distance and compression often make plates unreadable, which is the usual situation for plates in wide junction-camera views [10].

- **Advantages:** realistic surveillance data; multi-frame tracks of the same plate.
- **Disadvantages:** non-Indian plates; even the best methods leave many plates unreadable, so a reader must be able to abstain.

Table 2.1 compares the technique families on the criteria that matter most for an enforcement and signal-control platform.

Table 2.1: Comparison of the technique families

Technique family	Accuracy	Transparency	Relevance to TraffiSense
Adaptive signal control [1][2]	Gains over fixed timing	Medium (rules) to low (LLM)	PRAVAH controller in a Unity junction simulator
Emergency priority [3]	High with two cues	High	Emergency exemption and timed ambulance priority
Object detection [4]	High in real time	Low per frame	Shared vehicle detection pass (IDD-trained YOLO11s)
Indian datasets [5]	Large gains from local data	Not applicable	Indian vehicle classes in training
Two-wheeler violations [6][7][8][9]	High on curated images	Medium	Helmet and rider-count checks with a “not sure” band
Low-resolution plates [10]	Low on distant plates	High with per-character votes	READ / NOT SURE / UNREADABLE plate decisions

2.3 Summary of Literature Survey

The literature shows a clear shift from manual observation and fixed signal plans to deep-learning perception and adaptive control, but most systems still decide on single frames, solve one problem in isolation, and do not say when they are unsure. Table 2.2 summarises the surveyed works.

Table 2.2: Literature survey summary

SN	Technique / work	Author and year	Advantages and disadvantages
1	Machine-learning based adaptive traffic prediction and control using Edge Impulse	Tolani et al., 2025 [1]	**Advantages:** green times follow predicted density and arrival time. **Disadvantages:** needs upstream sensors; demand counted per vehicle.
2	LLMLight: large language models as traffic signal control agents	Lai et al., 2025 [2]	**Advantages:** better generalisation and interpretability than RL on ten datasets. **Disadvantages:** heavy model; lane-based benchmark networks.
3	LSTM and ResNet18 for ambulance routing and signal control	Alruwaili et al., 2025 [3]	**Advantages:** sirens and images both detect ambulances; Raspberry Pi signal priority. **Disadvantages:** priority on detection, regardless of the queue ahead.
4	YOLOv12: attention-centric real-time object detectors	Tian et al., 2025 [4]	**Advantages:** 40.6 % mAP at 1.64 ms on a T4 GPU. **Disadvantages:** generic training data; no Indian vehicle classes.
5	UVH-26 Indian traffic dataset and models	Sharma et al., 2025 [5]	**Advantages:** 26,646 CCTV images, 1.8 million boxes, 14 Indian classes. **Disadvantages:** single city; very large download.
6	Rider helmet violation detection with NVIDIA TAO and YOLOv8	Deshpande et al., 2025 [6]	**Advantages:** 98.56 % helmet and 97.6 % plate detection on 1,715 images. **Disadvantages:** single-image decisions; no “not sure” outcome.
7	Two-wheeler rider identification and triple-riding detection	Deshpande et al., 2025 [7]	**Advantages:** 91.42 % triple-rider and 98 % two-wheeler detection. **Disadvantages:** 81 % plate detection; single-frame decisions.
8	Triple-rider violations with an optimised Mask R-CNN	Saravanan and Rajini, 2025 [8]	**Advantages:** 97.93 % triple-rider detection across weather conditions. **Disadvantages:** heavy two-stage model; no plate reading.
9	DashCop: e-ticket generation from dashcam video	Rawat et al., 2025 [9]	**Advantages:** end-to-end e-tickets; rider–motorcycle association; RideSafe-400 dataset. **Disadvantages:** moving viewpoint; no human review.
10	ICPR 2026 competition on low-resolution plate recognition	Laroca et al., 2026 [10]	**Advantages:** 20,000 training and 3,000 test tracks of real surveillance plates. **Disadvantages:** non-Indian plates; many plates remain unreadable.

Research gaps. The survey exposes several gaps that motivate the TraffiSense design. First, most violation detectors decide on single frames, so glare, occlusion or a neighbouring rider can produce a wrong record. Second, systems rarely state when they are *not sure*; a forced decision on an unclear rider becomes a wrong challan. Third, number-plate readers return their best guess, although a guessed plate charges the wrong citizen. Fourth, the same vehicle is often detected many

times, by different checks or in a re-uploaded video, and no mechanism prevents several challans for one vehicle. Fifth, emergency vehicles are seldom exempted automatically. Sixth, explanation and evidence are usually limited to a confidence score. Seventh, adaptive signal controllers count vehicles rather than the people waiting, which suits Indian mixed traffic poorly. Table 2.3 maps these gaps to the corresponding TraffiSense design decisions.

Table 2.3: Research gaps and the corresponding TraffiSense design decisions

Research gap	TraffiSense design decision
Single-frame decisions	Shared detection and ByteTrack; every check votes over all frames of a tracked vehicle or rider.
No “not sure” outcome	Operating points chosen on held-out cameras for zero wrong calls; between thresholds a case goes to human review.
Guessed number plates	Per-character voting across frames and an Indian format check; only READ plates are printed.
Several challans for one vehicle	Union-find merging by track, plate, appearance and re-upload matching; unique index of one challan per case.
Emergency vehicles challaned	CLIP probe exempts ambulance, police and fire vehicles with a recorded reason.
Weak evidence and explanation	Three evidence frames, crop, clips, plain-language reasons and a SHA-256 fingerprint on every finding and challan.
Signal control counts vehicles, not people	PRAVAH weights waiting people and long red waits; evaluated in a simulator of an Indian junction.

Chapter 3

Implemented System: TraffiSense

3.1 Overview

This chapter presents TraffiSense, the evidence-first traffic enforcement platform that has been implemented in this project. TraffiSense is an academic prototype for analysing recorded traffic video. It is not an autonomous enforcement authority: its challans are produced only when a model is certain under strict, pre-measured rules, every other case is decided by an authorised reviewer, and every output carries the evidence needed to check it.

The implemented system integrates the following capabilities. An authentication module lets operators create an account with an e-mail address or phone number, verify it with a one-time code, sign in and reset a forgotten password. An analysis module accepts a video upload with a choice of checks, keeps the original file unchanged, and processes it in the background while the page shows live progress. A model layer detects and tracks vehicles once and runs the helmet, triple-riding, red-light and speed checks, the number-plate reader and the emergency-vehicle probe. An evidence module stores frames, crops and clips for every finding and fingerprints them. A decision module routes each finding, merges repeat detections of the same vehicle and issues e-challans with a PDF. Review, evidence, digital-twin, lane, analytics and settings pages complete the platform, and a signal-timing page runs the traffic measured in a video through a Unity junction simulator to compare signal controllers. All records are stored in an SQLite database . Table 3.1 summarises the implemented system.

Table 3.1: Summary of the implemented TraffiSense system

Aspect	Implementation
System type	Web-based, evidence-first traffic enforcement platform for recorded video (academic prototype).
User role	Operator (uploads video and reviews cases); every account is verified before first use.
Frontend	React 19, TypeScript and Vite; react-router; three.js for the 3-D digital twin; 18 pages.
Backend	FastAPI on uvicorn, port 8600, Python 3.14; serves the API and the built web app.
Model layer	Ultralytics YOLO11 detectors and classifiers on PyTorch (Apple GPU via MPS), ByteTrack, CLIP probe, ONNX plate reader.
Checks	Helmet, triple riding, red-light jumping, over-speeding; number plates; emergency exemption; lanes as context.
Evidence	Three frames, crop, original and annotated clips (H.264 via FFmpeg), SHA-256 fingerprint.
Database	SQLite in WAL mode: users, sessions, codes, analyses, findings, reviews, challans.
Decision principle	Precision first; uncertain cases to human review; at most one challan per vehicle.
Signal control (simulated)	Unity 6 junction simulator (C#) with four controllers including PRAVAH; Python TCP bridge for Python controllers; results always labelled SIMULATED.

The functional requirements derived from the project definition are: account creation, verification, sign-in and password reset; video upload with a selection of checks; background processing with progress; detection of the four offence types; number-plate reading; emergency-vehicle exemption; evidence collection; routing to automatic challan, review or exemption; human review with plate entry; e-challan PDF download; and evidence, twin, lane and analytics views. The main non-functional requirements are precision before recall, explainability of every decision, integrity of evidence, secure handling of passwords and sessions, recovery after a server restart, and a modular model layer in which a detector can be added or replaced without changing the rest of the platform.

3.1.1 Existing System Architecture

In the existing process, footage from CCTV cameras or phones is watched by an officer, either at the junction or on a control-room screen. When the officer notices an offence, it is noted from what was seen at that moment, the plate is read or written down by hand, and a challan is issued, usually with little recorded evidence and no explanation. Figure 3.1 shows this flow together with the limitation that arises at each stage.

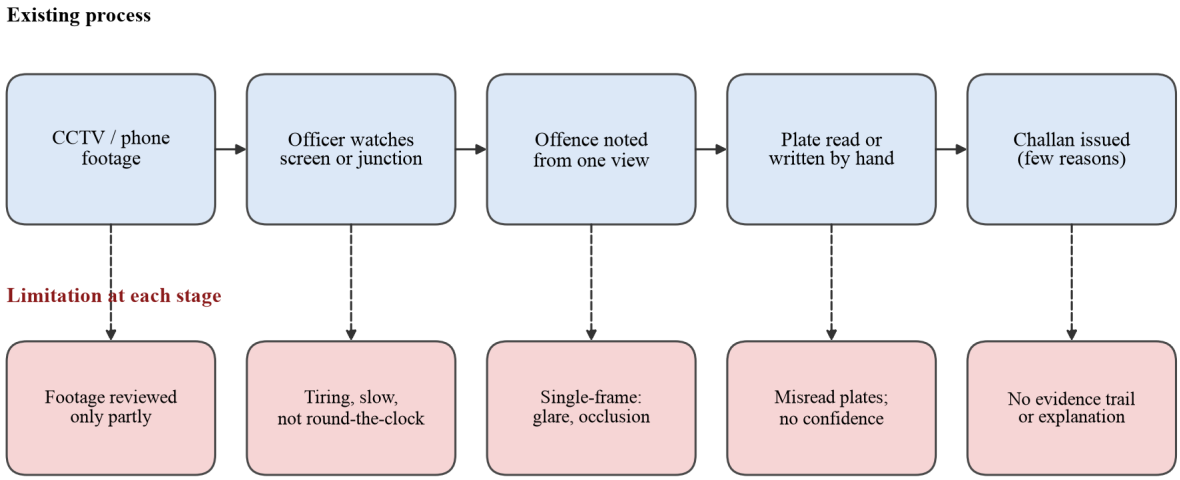

Fig 3.1: Existing enforcement flow and its limitations

Single-purpose enforcement cameras automate parts of the third and fourth stages, but they still decide on single frames, they cover one offence each, and they give the reviewer no account of why a vehicle was flagged. Table 3.2 lists the main limitations and the response taken in TraffiSense.

3.1.2 Proposed System Architecture

The proposed architecture of TraffiSense has three tiers and a model layer. Operators work through a **React frontend**, built with Vite and served by the same server. The frontend talks to the **FastAPI backend** on port 8600 through a JSON API; the backend provides authentication, the analyses API and file serving, findings, review and challans, and the statistics behind the dashboard, analytics and digital twin. Video analysis is delegated to a background **analysis pipeline**, which calls the trained tools of the **model layer** as separate processes. Persistent data live in **SQLite** and in one folder per analysis. Figure 3.2 shows the architecture.

Table 3.2: Limitations of the existing system and the response in TraffiSense

Limitation of existing systems	Response in TraffiSense
Manual watching; most footage never reviewed	Every uploaded video is processed completely in the background.
Decision from one glance or one frame	Tracked, multi-frame decisions with minimum frame counts and agreement checks.
Forced decisions on unclear cases	Explicit “not sure” band; uncertain cases go to the human review queue.
Misread or guessed plates	Plate printed only when READ with certainty; otherwise the reviewer confirms it.
Emergency vehicles caught by rules	Ambulance, police and fire vehicles exempted with a recorded reason.
Duplicate penalties	Duplicate merging and a unique index: one challan per vehicle.
No evidence trail or reasons	Frames, clips, reasons, confidence and SHA-256 fingerprint on every challan.

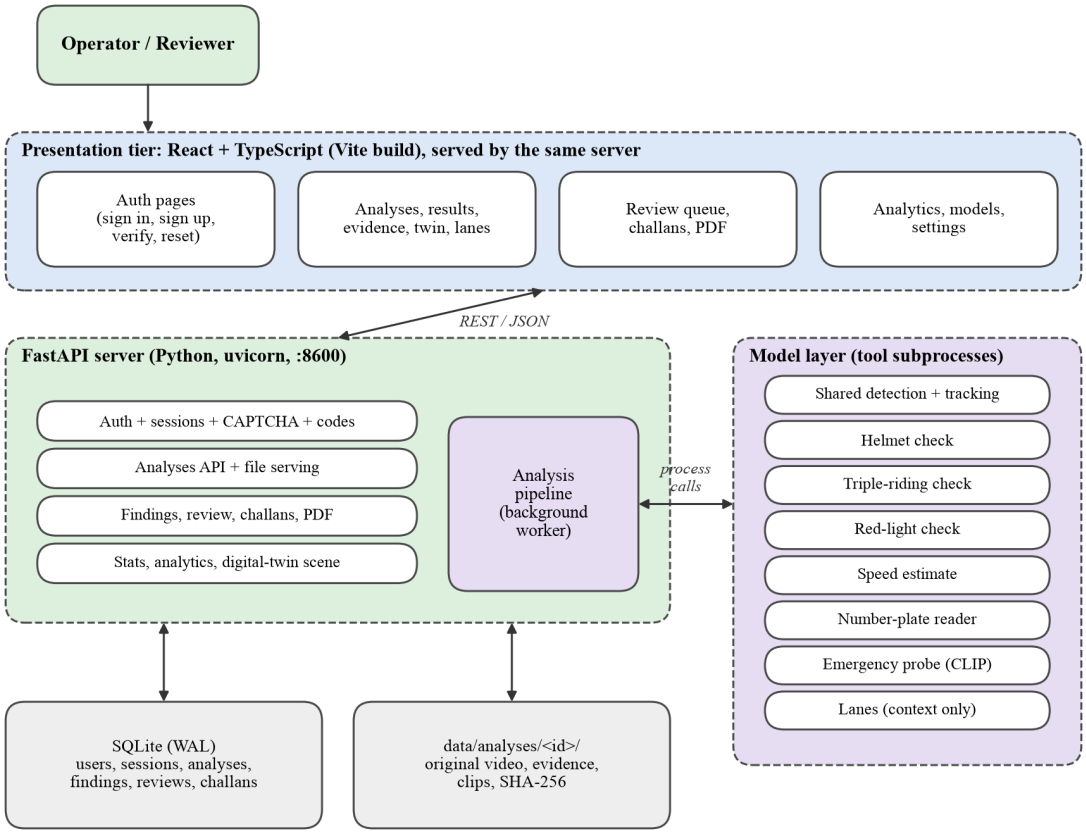

Fig 3.2: Proposed architecture of TraffiSense

Each trained tool runs as a subprocess with its own models and decision rules, and the pipeline only reads its JSON output. This keeps the web server responsive while the GPU is busy, isolates a failing tool, and lets a model be retrained or replaced by writing one *Detector* class. The deployment view in Figure 3.3 shows the processes that run when the platform is started with `./start.sh`.

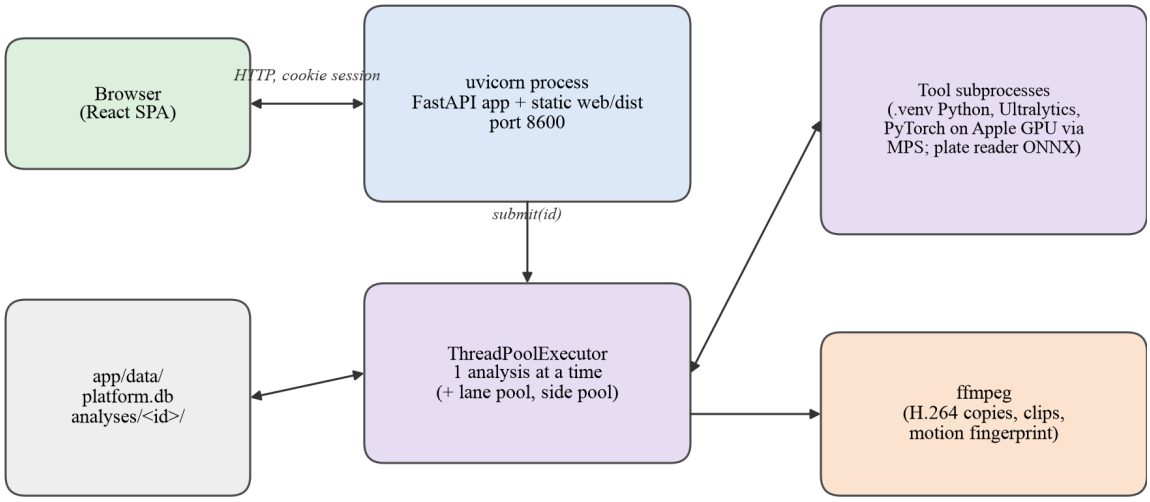

Fig 3.3: Runtime and deployment view of TraffiSense

The responsibilities of each component are summarised in Table 3.3.

The frontend has 18 pages, grouped by purpose in Figure 3.4: four public authentication pages and fourteen pages behind sign-in, which share one layout with a sidebar and a live count of running analyses and cases waiting for review. Table 3.4 lists each page and its purpose.

3.2 Implementation Details

This section describes how the platform is implemented: the methodology and techniques behind each check, the diagrams that describe the behaviour of the system, and the hardware and software used to build and run it.

Table 3.3: Components of the proposed architecture and their responsibilities

Component	Runtime	Responsibility
React frontend	Browser; built to web/dist	Pages for upload, results, review, challans, evidence, twin, lanes, analytics, settings.
FastAPI backend	uvicorn, port 8600	Authentication, sessions, CAPTCHA, verification codes, rate limits, API, PDF generation.
Analysis pipeline	Thread pool, one analysis at a time	Working copy, shared detection, checks, plates, evidence, merging, challans, lanes.
Model layer	Python subprocesses (MPS)	Detection and tracking, helmet, triple riding, red light, speed, plates, emergency probe, lanes.
Signal simulator	Unity 6, headless batch mode	Indian four-arm junction; runs every controller on identical arrivals; 3-D views for demonstration.
Signal bridge	Python TCP server, port 5055	Lets controllers written in Python run the simulated signal; PRAVAH and the project controller.
SQLite + files	data/platform.db, data/analyses/<id>/	Users, sessions, analyses, findings, reviews, challans; original video and evidence.

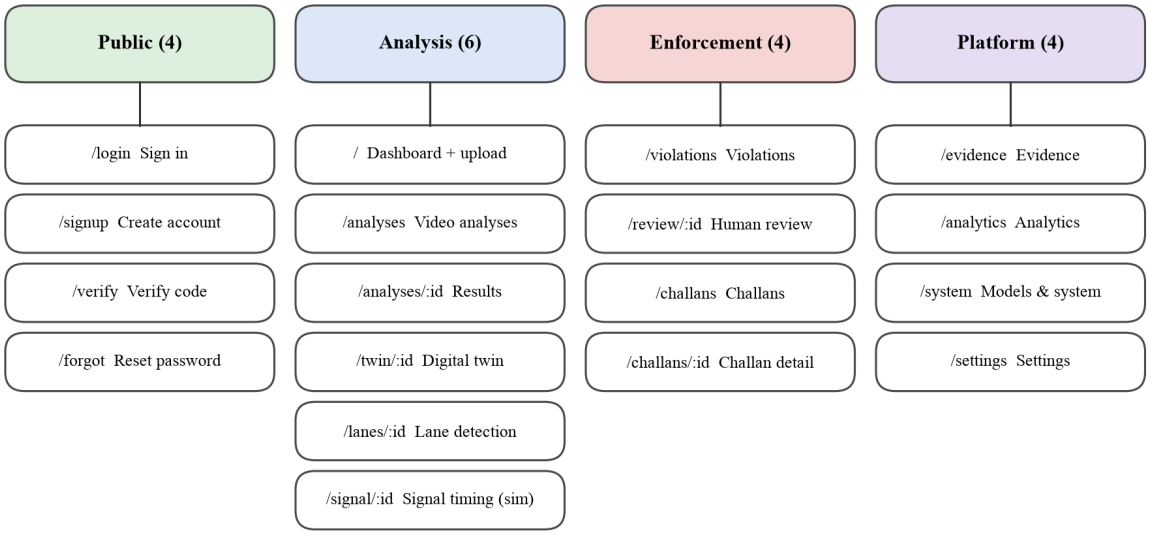

Fig 3.4: Map of the 18 frontend pages grouped by purpose

Table 3.4: Frontend pages of TraffiSense

Route	Group	Purpose
/login	Public	Sign in with e-mail or phone and password
/signup	Public	Create an account (name, contact, password policy, CAPTCHA)
/verify	Public	Enter the six-digit verification code
/forgot	Public	Request a reset code and set a new password
/	Analysis	Dashboard: totals, video guidance, upload, review queue preview
/analyses	Analysis	All video analyses with status, findings, reviews and challans
/analyses/:id	Analysis	Video player with overlays, results list, lane structure, notes
/twin/:id	Analysis	3-D digital twin of the scene with violations on a timeline
/lanes/:id	Analysis	Lanes estimated from vehicle movement (context only)
/signal/:id	Analysis	Signal timing (SIMULATED): measured traffic replayed through the junction simulator
/violations	Enforcement	Every finding with its decision path and filters
/review/:id	Enforcement	Human review: evidence, reasons, plate, decision
/challans	Enforcement	Issued challans with source, confidence and PDF
/challans/:id	Enforcement	Challan detail with evidence and download
/evidence	Platform	Gallery of evidence frames for all findings
/analytics	Platform	Violation types, outcomes, confidence distribution
/system	Platform	Models and their status, how decisions are made
/settings	Platform	Profile, password, theme, upload defaults, code delivery

3.2.1 Algorithm / Methodology / Techniques

An analysis follows the pipeline in Figure 3.5. The steps are described in the paragraphs below.

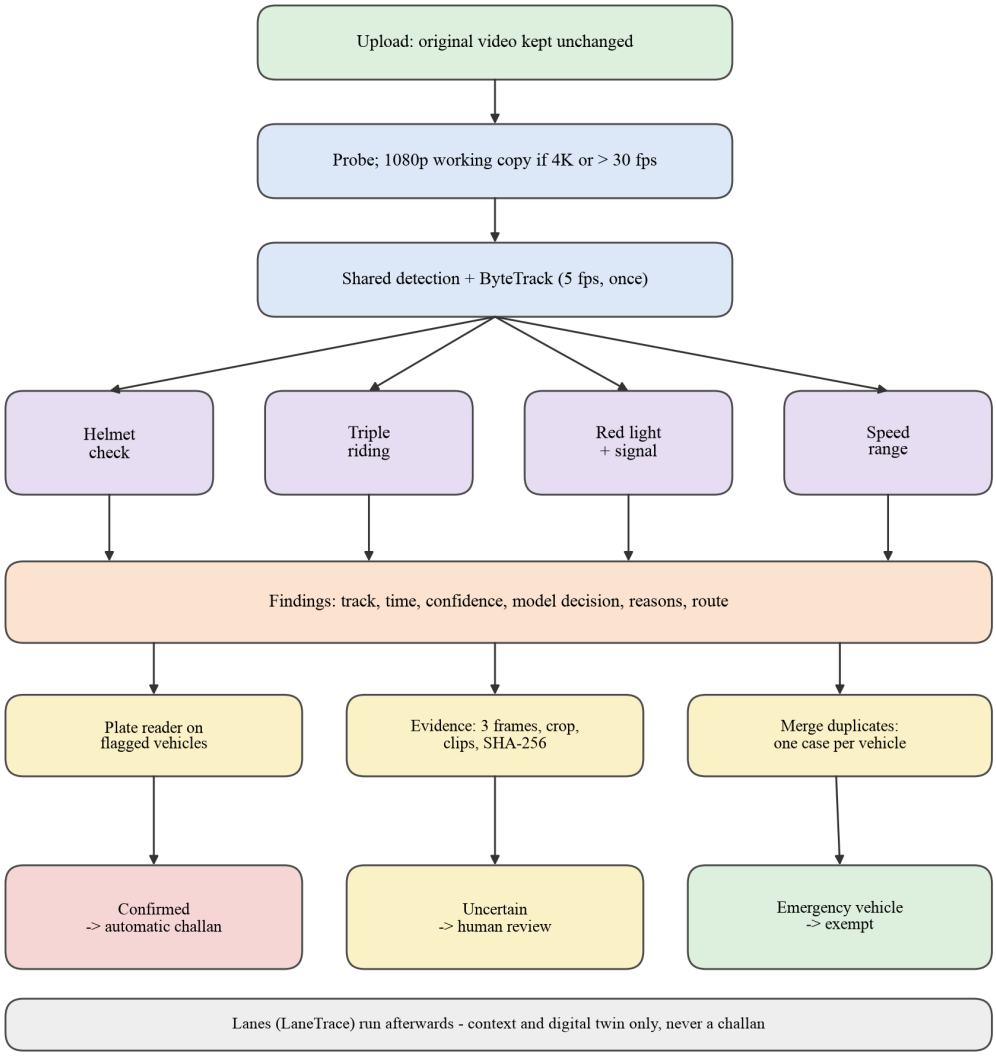

Fig 3.5: Video analysis pipeline of TraffiSense

A. Video intake and working copy

The uploaded file (up to 2,048 MB) is stored unchanged in `original/` and is never modified, so the evidence can always be traced back to it. The video is probed for resolution, frame rate and length. When it is larger than 1080p or faster than 30 frames per second, a 1080p working copy is made with FFmpeg and the models run on that copy, while a browser-playable H.264 copy is prepared in parallel. Frames are sampled at about 5 per second, so the sampling step for a video with frame rate f is

$$k = \max(1,\ \text{round}(f/5)) \tag{3.1}$$

B. Shared vehicle detection and tracking

A YOLO11s detector fine-tuned on the Indian Driving Dataset (input 960 px, confidence 0.3) finds cars, buses, trucks, auto-rickshaws, motorcycles, riders, persons and other classes in every sampled frame, and ByteTrack links the boxes into tracks. This pass runs **once** per video; the helmet, triple-riding and plate tools replay its cached detections through an import hook, so all checks see identical boxes and track numbers. Boxes are compared with the intersection over union,

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{3.2}$$

which is also used to link a finding to the tracked vehicle behind it (IoU $\geq$ 0.3) and to find neighbouring vehicles whose plates must be read together.

C. Helmet check

Only ridden or moving motorcycles that a second, COCO-trained detector also confirms are checked ; bicycles, parked bikes and look-alikes are skipped. A head counts only if it sits over the bike, which removes posters, murals and bystanders, and a rider re-found by the tracker is stitched back into one person. A YOLO11s classifier at 128 px labels each head crop as helmet, no helmet or not a head. The frame probabilities p_t of a rider are combined with weights w_t that grow with the head size:

$$\bar{p}_c = \frac{\sum_t w_t \, p_{t,\,c}}{\sum_t w_t}, \qquad c \in \{\text{helmet},\ \text{no helmet}\} \tag{3.3}$$

The rider is called **helmet** if the combined helmet probability is at least 0.90 and **no helmet** only if the combined no-helmet probability is at least 0.98 over at least three frames. If fewer than 85 % of the confident frames agree, or if the clearest view says the opposite, the answer is **not sure**. A confirmed no-helmet rider in the second or later seat is still sent to review, because a person standing next to the bike can be mistaken for a pillion rider.

$$\begin{aligned} d &= \text{no helmet} \quad \text{if } \bar{p}_{nh} \geq 0.98,\ n \geq 3,\ \text{agreement} \geq 0.85 \\ d &= \text{helmet} \quad \text{if } \bar{p}_h \geq 0.90 \\ d &= \text{not sure} \quad \text{otherwise} \end{aligned} \tag{3.4}$$

D. Triple-riding check

For every tracked motorcycle the crop of the bike with its riders is classified by a YOLO11s classifier (256 px) into one, two, three-or-more riders or not a bike, and the number of heads sitting on the seat is counted. Four features are combined by a logistic model fitted on held-out cameras: the logits of the three-or-more and two-rider probabilities, the share of frames with at least three heads on the seat, and the maximum head count divided by four. With standardised features $z_j = (x_j - \mu_j) / \sigma_j$,

$$s = \sigma\left(b + \sum_{j=1}^{4} w_j z_j \right), \qquad \sigma(u) = \frac{1}{1 + e^{-u}} \tag{3.5}$$

A motorcycle is called **triple riding** only if $s \geq 0.897$ and at least three frames agree; between 0.839 and 0.897 it is **not sure** and goes to review. The thresholds were chosen for zero wrong triple calls on held-out cameras with at most about one bike in twelve marked not sure.

E. Red-light check

Signal heads are found by the detectors and by a colour finder that looks for red and green lamps next to each other that are never lit together. A YOLO11s classifier at 64 px labels each lamp crop as red, yellow, green or not a light. Because a lamp keeps its colour for seconds and changes in a fixed order, the per-frame probabilities are smoothed over time in the manner of a hidden Markov model: with transition matrix A and frame likelihoods b_t, the filtered state is

$$\alpha_t(j) \propto b_t(j) \sum_i \alpha_{t-1}(i)\, A_{ij} \tag{3.6}$$

The controlled direction and the stop line are found automatically: the stop line is placed where the front of the queue of vehicles that came this way stands during red. A vehicle commits a violation when its ground point crosses the stop line in the controlled direction while the smoothed signal has been red for at least one second, both at the last sighting before the line and at the first after it. A challan additionally requires that the head was **seen changing colour** in the shot, so a red signboard or a red vehicle can at most cause a review. Just-turned-red cases, weak signal readings and short tracks go to review.

F. Emergency-vehicle exemption

Each flagged vehicle crop is embedded with a CLIP ViT-B/16 image encoder, and a logistic probe trained with scikit-learn gives the probabilities of ambulance, police, fire and other. A vehicle is **exempt** if the emergency probability is at least 0.956 and is sent to review if it is at least 0.541. Exempt findings never become challans, and the reason is stored with the finding.

G. Speed estimate

Speed is checked only when a one-time calibration (a known road length or standard lane dashes) and a speed limit are supplied. Box bottoms are mapped to the road plane using the camera geometry, and a velocity $v = \Delta d / \Delta t$ is fitted per vehicle. The uncertain inputs (lens, vehicle size, tracking noise, model error) are varied in about 300 Monte-Carlo repetitions, in the spirit of the bootstrap, and the 90 % range is taken from the 5th and 95th percentiles:

$$[\, v_{lo},\ v_{hi} \,] = [\, Q_{0.05}(v^{(1..M)}),\ Q_{0.95}(v^{(1..M)}) \,] \tag{3.7}$$

Over-speeding is confirmed only if v_{lo} is above the limit and the confidence is high or medium; if the range includes legal speeds, the case is reviewed.

H. Number-plate reading

For the vehicles flagged by a check, and the vehicles overlapping them, a YOLO plate detector finds the plate in every frame, and a fast-plate-ocr reader fine-tuned on Indian plates and exported to ONNX reads it. Characters are voted across frames, weighting each reading by its character confidence, and the result is checked against the Indian registration format (for example MH12AB1234). The decision is **READ** when two or more sharp frames agree without a sharp disagreement, **NOT SURE** when a reading exists but is not certain, and **UNREADABLE** otherwise. If the same plate is read on two vehicles, it is kept only on the vehicle with at least twice as many agreeing reads; otherwise all become NOT SURE. Only READ plates are printed; a NOT SURE reading is shown to the reviewer to confirm or correct.

I. Routing, evidence and challan issue

Each finding is routed as shown in Figure 3.6. For every finding the pipeline saves the frames at $t - 0.6$ s, t and $t + 0.6$ s (raw and with the box drawn), a close-up crop, a clip of the original video and a clip of the tool's annotated video, and the plate crop when available. The evidence fingerprint is the SHA-256 hash over the names and contents of all evidence files $e_1, \ldots, e_n$ in sorted order:

$$H = \mathrm{SHA256}(\mathrm{name}_1 \,\|\, \mathrm{bytes}_1 \,\|\, \cdots \,\|\, \mathrm{name}_n \,\|\, \mathrm{bytes}_n) \tag{3.8}$$

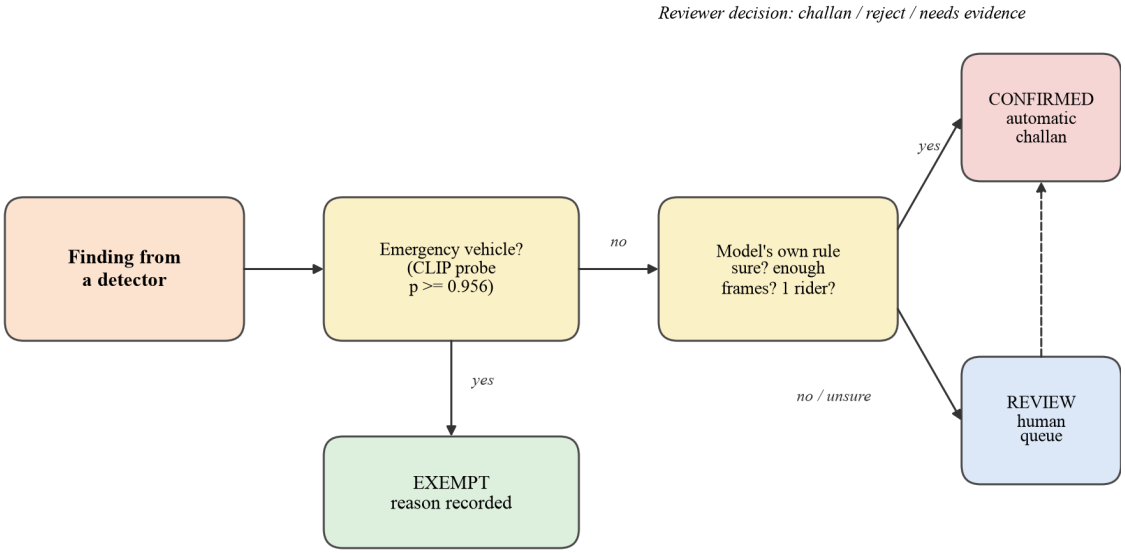

Fig 3.6: Routing of a finding to challan, review or exemption

A confirmed finding becomes an automatic challan numbered `TVP-YYYYMMDD-XXXXXX`, which stores the offences charged, the plate (a READ plate or the plate typed by the reviewer) and the evidence hash. A reviewer can issue a challan, reject the case or mark that more evidence is needed. The challan PDF is generated with ReportLab and contains the vehicle and offence details, the explanation, the evidence frames and crops, a QR code with the challan number and hash, and the fingerprint in the footer.

J. One case per vehicle

The same vehicle can be flagged several times: by two checks, for two riders, after the tracker loses and re-finds it, or in a second upload of the same footage. TraffiSense groups such detections with a union–find structure (Figure 3.7). Two findings are joined when they belong to the same shared track or to tracks that the tracker re-found within one second at the same place and size, when the same plate was READ, when the same violation occurs within 8 s with near-identical close-ups (CLIP cosine similarity ≥ 0.92), or when another upload of the same footage shows the same vehicle at the same moment.

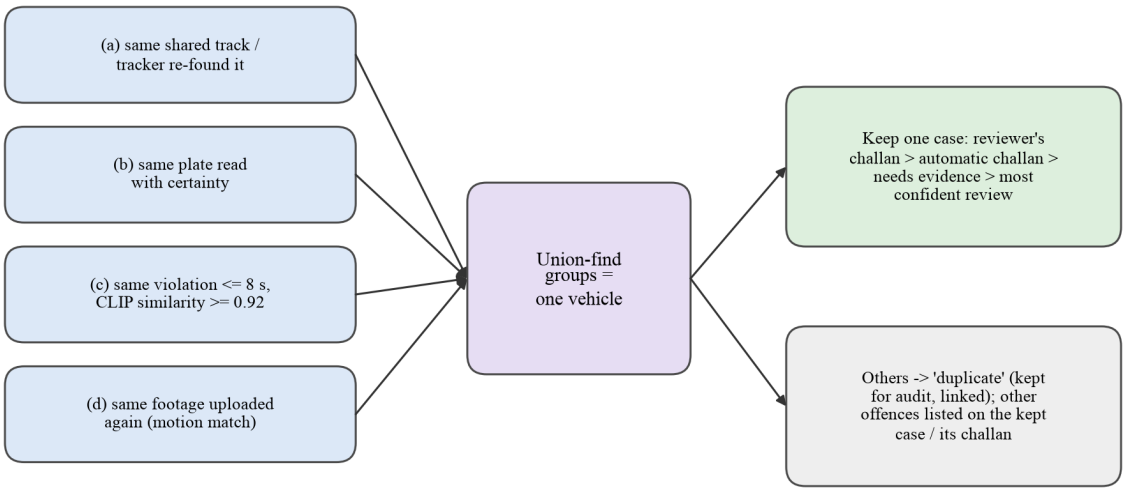

Fig 3.7: One case per vehicle: merging repeat detections

Re-uploads are recognised from a motion signature: each video is reduced to 64 × 36 grey frames at 5 per second, frame differences d_t are normalised, and the best alignment k between two videos maximises the average correlation of their motion,

$$k^{*} = \arg\max_{k} \frac{1}{|O_k|} \sum_{t \in O_k} \hat{d}_t^{A} \cdot \hat{d}_{t+k}^{B}, \qquad \text{accepted if score} \geq 0.5 \tag{3.9}$$

Motion is compared instead of the picture itself, because the same fixed camera on another day shows the same road but never the same moving traffic. In each group one case is kept, in the order: a reviewer's challan, an automatic challan, a case waiting for more evidence, then the most confident case waiting for review. The others become *duplicate* records that remain linked for audit, and a different offence of the same vehicle is added to the kept case and, when certain, to its

challan. A unique database index guarantees at most one challan per case. When an analysis is deleted, the findings merged into its cases are restored and re-merged.

K. Lanes and digital twin

LaneTrace estimates lanes from how vehicles actually move during the first 30 seconds, ignoring painted markings, which are often faded in India; camera movement and scene cuts are detected. Lanes are **context only**: they support the evidence view and the digital twin and never create a challan. The digital twin rebuilds the scene in 3-D with three.js from the shared tracks: vehicles are placed on the road plane using the camera horizon, coloured by class, and violations are marked on a timeline.

L. Adaptive signal control in simulation (PRAVAH)

Signal timing cannot be tested on live roads, so it is developed and evaluated in a microscopic simulator of an Indian four-arm junction built in Unity 6. Its core is a headless model with a 0.1 s step: vehicles follow the Intelligent Driver Model, drive on the left and arrive with an Indian mix (38 % two-wheelers, 28 % cars, 16 % autorickshaws, 8 % trucks, 6 % buses, 4 % bicycles) weighted in passenger-car units (PCU). Two-wheelers take lateral positions across the lane and creep past the stop line, each driver has a patience after which they jump the red, left turns are free, buses and autos stop just past the junction, a vehicle stuck in the junction box blocks cross traffic, and stray cattle occasionally block the kerb lane. A 3-D view with day, night and monsoon lighting and eight camera modes is used for demonstration.

PRAVAH (Person-weighted, Red-patience-Aware, Vehicle-mix Adaptive Headway control) is the signal controller developed in this project. It serves one approach at a time and uses only what a junction camera sees within 100 m of the stop line. A person who has waited w seconds contributes an urgency

$$U(w) = 1 + \left(\frac{w}{\tau}\right)^2, \qquad \tau = 75 \text{ s} \tag{3.10}$$

so that long red waits, which lead to red-light jumping, become expensive quickly. Each waiting vehicle counts its typical occupants (about 40 for a bus, 1.4 for a two-wheeler). The time needed to clear approach a and the gain of serving it next are

$$C_a = 2 + \frac{Q_a}{\hat{s}_a}, \qquad G_a = \frac{\sum_{i \in a} p_i \, U(w_i) \, \varphi(o_a)}{C_a + L} \tag{3.11}$$

where Q_a is the queued load in PCU, $\hat{s}_a$ the discharge rate, initialised from the Indian Roads Congress saturation flow of 525 PCU/h per metre of approach width and learned online from the vehicles that actually cross during green, p_i the occupants, L = 5 s the amber and all-red loss, and $\varphi(o) = \mathrm{clip}(1 - (o - 0.5)/0.4, 0, 1)$ a factor that discounts an approach whose exit is more than half full, so that the controller does not feed a blocked junction box. After its planned queue has cleared, a green continues only while it relieves more urgency per second than the best other approach would; any approach that has waited 90 s is served next. For an ambulance, the switch is timed so that the queue ahead of it has cleared when it arrives.

Four controllers are compared on identical arrivals: a fixed 30 s rotation, a fixed-time plan designed with Webster's method from the true flows, a queue-pressure adaptive controller (Adaptive v1) and PRAVAH. A TCP bridge on port 5055 lets controllers written in Python, including a Python port of PRAVAH and the project's own rule-based signal controller, run the simulated signal. On the */signal/:id* page, the vehicles counted and classified in an analysed video, split by direction of travel, become the arrivals of one approach, the operator supplies assumptions for the other three, and the simulator runs every controller headless on a private copy of the project. Every number on this page is labelled SIMULATED.

M. Authentication and security

Passwords must have at least 10 characters with upper- and lower-case letters, a digit and a symbol, and are stored as PBKDF2-HMAC-SHA256 hashes with a 16-byte random salt and 310,000 iterations. Sign-up is protected by a self-hosted CAPTCHA whose answer is bound to a signed token that expires after 5 minutes and can be used once. Accounts are activated by a six-digit code that is stored only as a keyed hash, expires after 10 minutes and allows at most 5 attempts. Sessions use an opaque random token in an HttpOnly cookie; only its HMAC is stored, and sessions expire after 7 days. Sensitive endpoints are rate-limited with a sliding window, and every API call checks the session on the server.

3.2.2 Use Case Diagram / Activity Diagram / DFD

Use case diagram. Figure 3.8 shows the use cases of TraffiSense. The operator signs up, verifies the account and signs in, uploads a video with a choice of checks, follows its progress, views the results, opens the evidence, twin and lane views and downloads challan PDFs. In the reviewer role the same user works through the review queue, confirms or types a plate and issues a challan, rejects the case or asks for more evidence. The model layer is a system actor that performs the analysis. Table 3.5 maps actors to use cases.

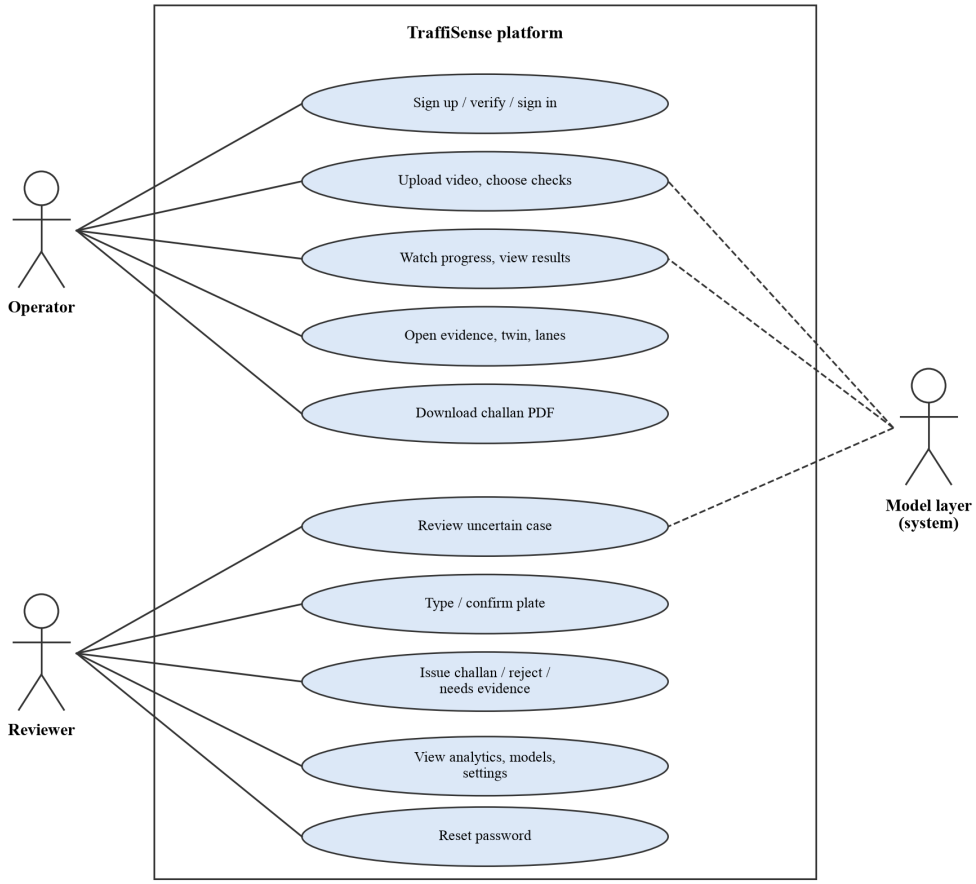

Fig 3.8: Use case diagram of the TraffiSense platform

Table 3.5: Mapping of actors to use cases

Actor	Use cases
Operator	Sign up, verify, sign in, reset password; upload a video and choose checks; follow progress; view results, evidence, digital twin and lanes; download challan PDFs; view analytics and settings.
Reviewer (operator in review role)	Open the review queue; inspect frames, clips, reasons and the plate crop; confirm or type the plate; issue a challan, reject, or mark that more evidence is needed.
Model layer (system)	Detect and track vehicles; run the selected checks; read plates; recognise emergency vehicles; estimate lanes.

Data flow diagrams. The level-0 DFD in Figure 3.9 treats TraffiSense as a single process that exchanges data with the operator and the reviewer, uses the trained models and stores data in SQLite and the evidence folders. The level-1 DFD in Figure 3.10 decomposes the platform into six processes and four data stores, as listed in Table 3.6.

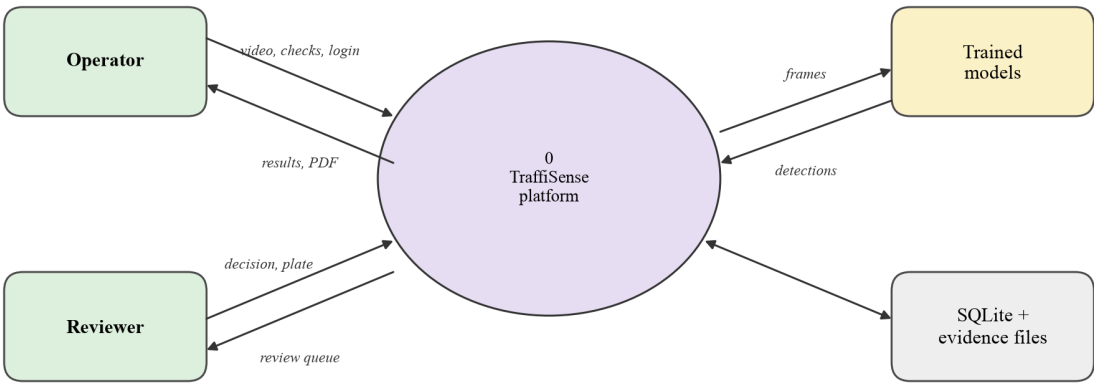

Fig 3.9: Level-0 data flow diagram

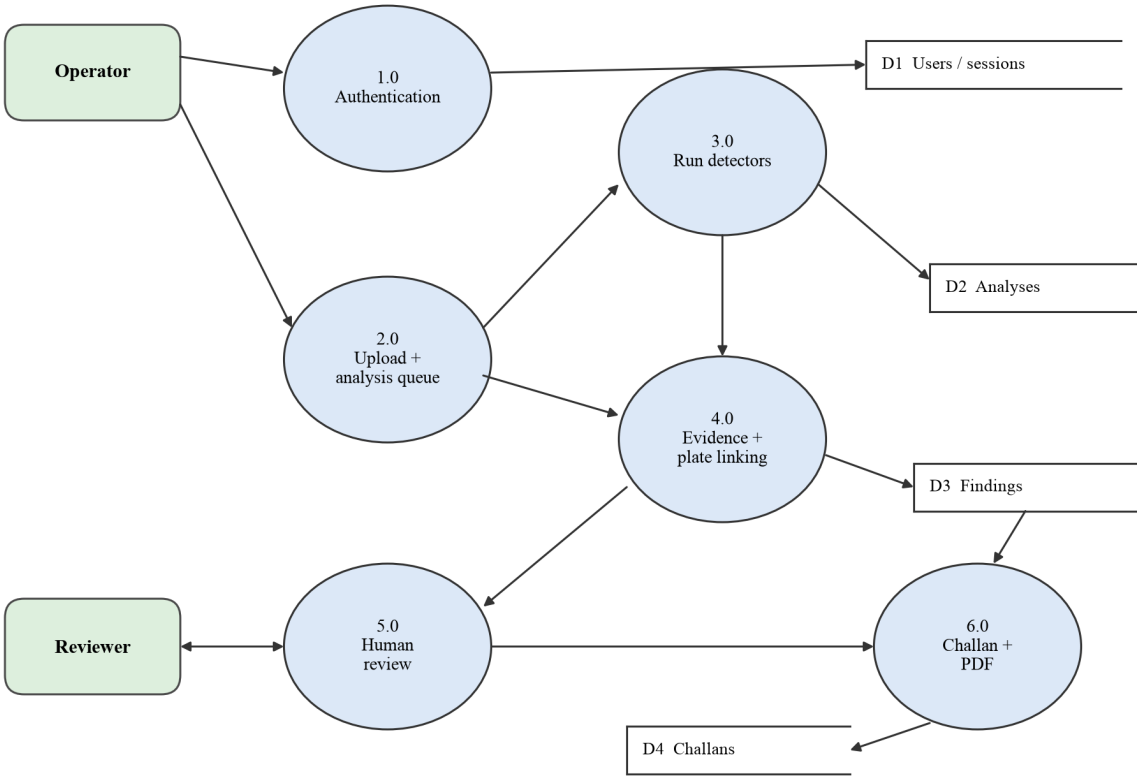

Fig 3.10: Level-1 data flow diagram

Activity diagram. The activity diagram in Figure 3.11 follows a video from upload to the final outcome across the operator, platform and reviewer lanes. After evidence is collected and duplicates are merged, the flow splits: a certain finding becomes an automatic challan, and an uncertain one goes to the reviewer, whose decision is recorded.

Sequence of an analysis. Figure 3.12 shows the message sequence of an analysis. The browser posts the video to `POST /api/analyses`; the server stores it, inserts the analysis row and submits it to the worker, then returns its id. While the browser polls the analysis, the worker runs the shared detection and the checks, reads plates for flagged vehicles, saves evidence and findings, merges duplicates and issues challans. Finally the browser can download a challan PDF. Table 3.7 lists the main API endpoints.

Table 3.6: Processes and data stores of the level-1 DFD

ID	Name	Description
1.0	Authentication	Sign-up with CAPTCHA, verification codes, sign-in, sessions, password reset.
2.0	Upload and analysis queue	Stores the original video, records options, queues the analysis.
3.0	Run detectors	Shared detection and tracking; helmet, triple-riding, red-light and speed checks.
4.0	Evidence and plate linking	Evidence frames and clips, plate reading, fingerprint, duplicate merging.
5.0	Human review	Presents uncertain cases; records the reviewer's decision, note and plate.
6.0	Challan and PDF	Issues one challan per vehicle and generates the e-challan PDF.
D1	Users / sessions	Accounts, hashed passwords, session hashes, verification codes.
D2	Analyses	Uploads, options, status, progress, metadata and artefacts.
D3	Findings	Findings with route, status, reasons, evidence and links to merged duplicates.
D4	Challans	Challan number, source, reviewer, plate, offences and evidence hash.

Step-by-step workflows. The operator workflow is:

1. Create an account, solve the CAPTCHA and enter the six-digit code; then sign in.
2. On the dashboard, read the guidance for a good video and drop a video file; choose the checks (no helmet, triple riding, red light; speed with a calibration).
3. Follow the live stage and progress bar while the analysis runs in the background.
4. Open the results: play the video with overlays, read each finding's reasons and confidence, and open its evidence.
5. Download the PDF of any automatic challan, and open the digital twin and lane views for context.

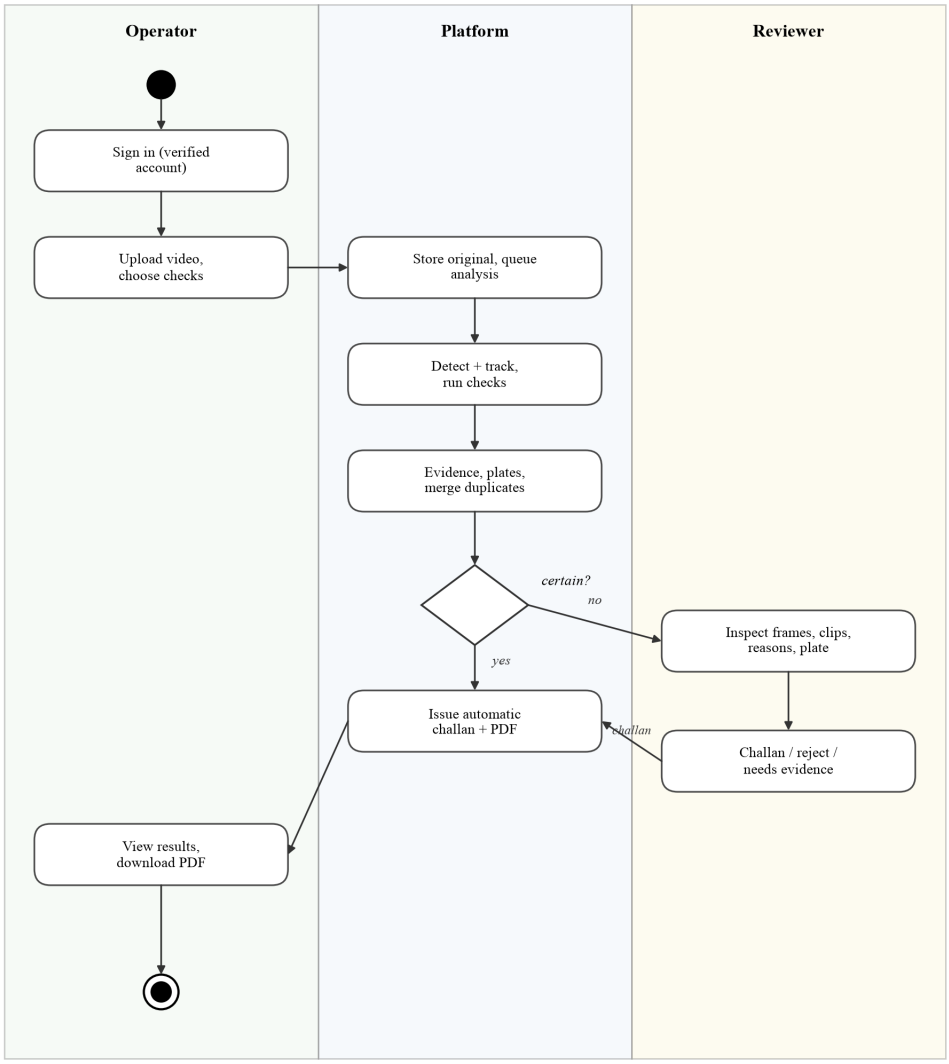

Fig 3.11: Activity diagram of the video analysis workflow

The review workflow is:

1. Open the human review queue, ordered with the most confident cases first.
2. Inspect the evidence frames, clips and the model's reasons, including why the case was uncertain.
3. Confirm or type the number plate from the plate crop if it was not read with certainty.
4. Decide: issue a challan (choosing the offences to charge), reject the case, or mark that more evidence is needed. The decision and note are recorded.

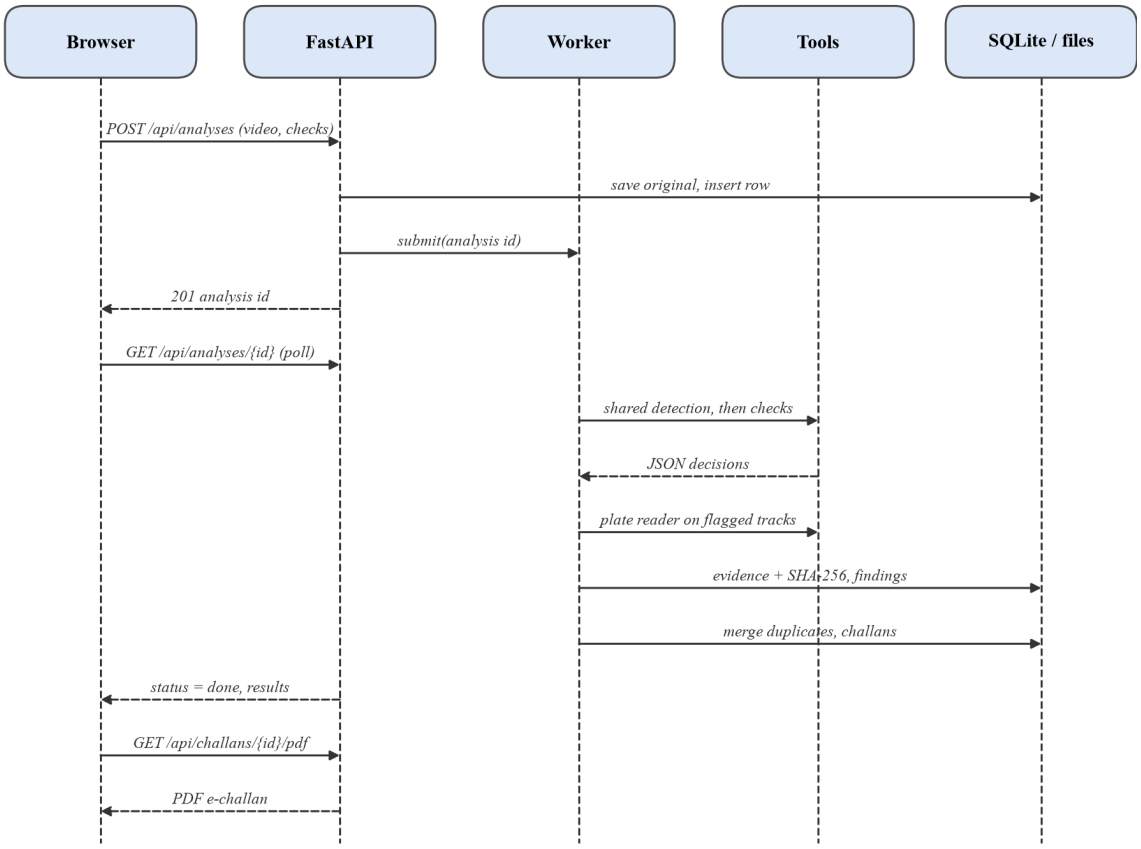

Fig 3.12: Sequence diagram of a video analysis

3.2.3 Hardware and Software Specifications

TraffiSense is developed and run on an Apple Silicon Mac. The models run on the Apple GPU through PyTorch's MPS backend; heavier training jobs were also run on Kaggle. Table 3.8 gives the hardware and Table 3.9 lists the software.

Table 3.7: Main API endpoints of TraffiSense

Endpoint	Purpose
GET /api/auth/captcha; POST /api/auth/signup	New CAPTCHA; create an account and send a verification code
POST /api/auth/verify, /resend, /login, /logout	Verify the code; resend it; sign in (session cookie); sign out
POST /api/auth/forgot, /reset	Send a reset code; set a new password
POST /api/analyses; GET /api/analyses[/{id}]	Upload a video with options; list analyses or read one with its findings
DELETE /api/analyses/{id}	Delete an analysis; merged findings elsewhere are restored
GET /api/analyses/{id}/files/{path}; /twin	Serve evidence and videos; digital-twin scene and tracks
GET /api/findings[/{id}]; POST /api/findings/{id}/review	List or open findings; record a review decision
GET /api/challans[/{id}]; /api/challans/{id}/pdf	List and open challans; download the e-challan PDF
GET /api/stats, /api/analytics, /api/system, /api/plates	Dashboard counts, analytics, model status, plate results
GET, POST /api/analyses/{id}/signal-sim	Traffic measured in the video and the last simulation; start a simulated signal-timing comparison

Table 3.8: Hardware specification

Item	Specification
Processor / GPU	Apple M4 (CPU, integrated GPU via Metal / MPS, media engine for H.264)
Memory	16 GB unified memory
Storage	Space for the virtual environment, model weights, datasets and per-analysis evidence folders
Training (additional)	Kaggle notebooks (CPU and T4 GPU) for longer training runs
Network	Needed only for package installation and for e-mail delivery of verification codes

Table 3.9: Software specification

Category	Software
Operating system	macOS 26
Languages / runtimes	Python 3.14 (backend and tools), Node.js 24 and npm (frontend build)
Frontend	React 19, TypeScript 6, Vite 8, react-router 7, three.js, lucide-react
Backend	FastAPI 0.141, uvicorn, Pydantic, SQLite (stdlib sqlite3), Pillow, ReportLab 5
Deep learning and vision	PyTorch 2.14 (MPS), Ultralytics 8.4 (YOLO11, ByteTrack), OpenCV 5, open_clip, scikit-learn
Plate reader	fast-plate-ocr (fine-tuned), ONNX Runtime
Media	FFmpeg (H.264 via VideoToolbox, software fallback)
Traffic simulation	Unity 6 (6000.6.4f1) with the Universal Render Pipeline, C#; CC0 sky and road textures
Version control	Git

The platform is configured through environment variables; no secret is stored in the code. Table 3.10 lists the configuration parameters.

Table 3.10: Configuration parameters

Parameter	Default	Purpose
PORT	8600	Port of the web server
TVP_DATA	app/data	Folder of the database and analyses
TVP_PYTHON	.venv/bin/python	Python used to run the tools
TVP_SMTP_HOST / USER / PASS / FROM	unset	E-mail delivery of codes; unset = developer mode
SESSION_DAYS	7	Session lifetime
MAX_UPLOAD_MB	2048	Maximum upload size
TVP_DEVICE	mps (if available)	Device for the models
TVP_UNITY	newest Unity in Unity Hub	Unity editor used to run the junction simulator headless

The platform is started with one command. The script builds the web app if needed and starts the server:

```
# one-time setup (project root)
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
cd app/web && npm install && npm run build

# start the platform -> http://127.0.0.1:8600
./app/start.sh

# frontend development (proxies /api to :8600)
cd app/web && npm run dev
```

Without SMTP settings, verification codes are shown in a clearly marked developer banner and written to `data/dev_outbox.log`, which allows the platform to be demonstrated offline.

The junction simulator runs headless from the command line, which is also how the platform starts it. Controllers written in Python are served through the bridge:

```
# compare all controllers -> signal_sim/Results/compare.md
Unity -batchmode -nographics -projectPath signal_sim \
      -executeMethod SignalSim.Experiment.Run

# Python controller through the TCP bridge (port 5055)
python3 signal_bridge/server.py --controller pravah

# robustness study -> signal_sim/Results/robustness.csv
Unity -batchmode -nographics -projectPath signal_sim \
      -executeMethod SignalSim.Robustness.Run -seeds 10
python3 signal_sim/Tools/robustness_report.py
```

Chapter 4

Result and Discussion

4.1 Sample of Inputs / Dataset / Database Used / Outputs / Screen Shots

This section describes the data that flow through TraffiSense: the videos and options supplied by the operator, the datasets used to develop the models, the database in which results are stored, the outputs produced for each finding, and the screens through which these are presented.

4.1.1 Sample Inputs

The main input is a recorded traffic video (MP4, MOV, AVI, MKV or WEBM, up to 2,048 MB). The dashboard shows the guidance in Table 4.1, because the quality of the video decides how many cases can be settled automatically and how many must be reviewed.

Table 4.1: Guidance for input videos shown on the dashboard

Property	Recommendation and reason
Fixed camera	CCTV pole or tripod; hand-held or moving footage breaks vehicle tracking.
One continuous shot	No cuts or compilations; every cut restarts tracking and short shots cannot be judged.
720p or better, sharp	1080p is ideal; riders' heads should be at least 40 px tall for helmet checks.
Angle 4–8 m high, looking along the road	Tilted down 20–45°; avoids occlusion between riders and keeps plates visible.
Daylight works best	At night lamps glare and fewer vehicles are detected, so more cases go to review.
20 seconds or longer	Each vehicle is judged over many frames, so it needs to stay in view for a few seconds.
Red light / speed	Red light is automatic (signal must be visible and seen changing colour); speed needs a calibration and a limit.

With the video, the operator selects the checks and options listed in Table 4.2. Defaults can be saved on the settings page.

Table 4.2: Options of an analysis

Option	Type	Description
checks	List	Any of helmet, triple, redlight, speed.
max_sec	Number	Analyse only the first N seconds (for example the first 2 minutes).
plates	Boolean	Read number plates of flagged vehicles (default on).
lanes	Boolean	Estimate lane structure afterwards (context only, default on).
speed_calib, speed_limit	File, number	Camera calibration and limit in km/h; speed is skipped without them.
camera	File (optional)	Manual stop line and direction for the red-light check; automatic otherwise.

4.1.2 Dataset Used

Each model was trained and evaluated on the data in Table 4.3. Wherever possible, scores come from cameras or whole videos that the model never saw during training (leave-cameras-out or leave-videos-out cross-validation), and downloaded or staged sets were used for training only. Some labels for the helmet and triple-riding sets were produced visually with AI assistance and have not yet been re-checked by a person; this is noted as a limitation.

4.1.3 Database Used

All records are stored in one SQLite database (`data/platform.db`) in write-ahead-logging mode, which allows the pages to read while the worker writes. Evidence files are kept outside the database in one folder per analysis (`data/analyses/<id>/`), next to the untouched original video. Table 4.4 lists the tables.

Table 4.3: Datasets used for model development and testing

Model	Data	Size and split
Vehicle detector	IDD-Detection (Indian Driving Dataset, YOLO export)	33,569 train / 4,196 val / 4,197 test images; YOLO11s at 960 px, 35 epochs (10.7 h)
Helmet classifier	28 Indian traffic videos (15+ cities, day/night/rain, CCTV/street/dashcam) + public head crops	4,212 persons (1,228 helmet, 873 no helmet, 1,265 can't tell, 846 not a head); 5-fold leave-videos-out; 7 unseen test videos (367 riders)
Triple-riding classifier	Police-camera clips, IDD stills, own videos; Kaggle and Roboflow sets for training only	114 real triple tracks; held-out test of 980 bikes with 125 triples, leave-cameras-out
Signal-state classifier	LISA Traffic Light, traffic-violation dataset, 216 Indian signal crops	Held-out LISA/TVD clips; Indian crops split in two halves, each tested by a model that never saw it
Emergency probe	Vehicle tracks from Indian clips (ambulance, police, fire) and ordinary traffic	170 emergency tracks (103 ambulance, 20 police, 47 fire) and 2,450 ordinary tracks held out
Plate reader	Indian vehicle licence-plate dataset (box + text) + degraded copies	1,696 plates; train 1,381 / test 315, grouped by plate text; 3,636 augmented training crops
Speed estimator	Two reference scenes with known speeds (M6, Autobahn); VS13 clips	137 vehicles with ground truth; 29 uncalibrated clips

Table 4.4: Tables of the TraffiSense database

Table	Main fields	Purpose
users	id, name, email, phone, pw_hash, verified, role, created	Accounts (password stored only as PBKDF2 hash)
sessions	token_hash, user_id, created, expires	Active sessions (only the HMAC of the token)
codes	target, purpose, code_hash, expires, attempts, used	Verification and reset codes
analyses	id, user_id, filename, status, stage, progress, options, meta	One row per uploaded video
findings	detector, violation, track, t, confidence, model_decision, route, status, reasons, uncertain_reasons, evidence, offences, duplicate_of	One row per detection, with evidence and links
reviews	finding_id, user_id, decision, note, created	Reviewer decisions
challans	number, finding_id, source, reviewer_id, status, plate, offences, evidence_hash	Issued challans (unique per case)

4.1.4 Outputs

Each analysis produces the outputs listed in Table 4.5. Together they answer what was detected, why, how sure the model was, and what happens next.

Table 4.5: Outputs of an analysis

Output	Description
Findings	Violation, vehicle, time, confidence, model decision and route for each detection.
Reasons	Plain-language evidence from the model, e.g. “no-helmet probability 0.99, vote over 14 frames”.
Uncertain reasons	Why a case went to review, e.g. “seen in only 2 frames” or “just turned red”.
Evidence	Frames before, at and after the moment (raw and boxed), crop, original clip, annotated clip, plate crop.
Plate result	READ with confidence, NOT SURE with a candidate for the reviewer, UNREADABLE or NOT LINKED.
Evidence fingerprint	SHA-256 over all evidence files of the finding.
Challan	Number, offences, plate, source (automatic or reviewer), evidence hash; PDF e-challan.
Annotated videos	Helmet, triple-riding, signal and lane overlays, playable in the browser.
Notes	Checks skipped or not possible (for example no visible signal), duplicates merged.

4.1.5 Screen Shots

The screens below show the main pages of the implemented system, captured from the running platform with the team's own analyses. Each figure is paired with a short description of what the page presents.

Figure 4.1 shows the sign-in and account-creation pages. Accounts are created with an e-mail address or phone number, a password that meets the policy and a CAPTCHA, and are verified with a six-digit code.

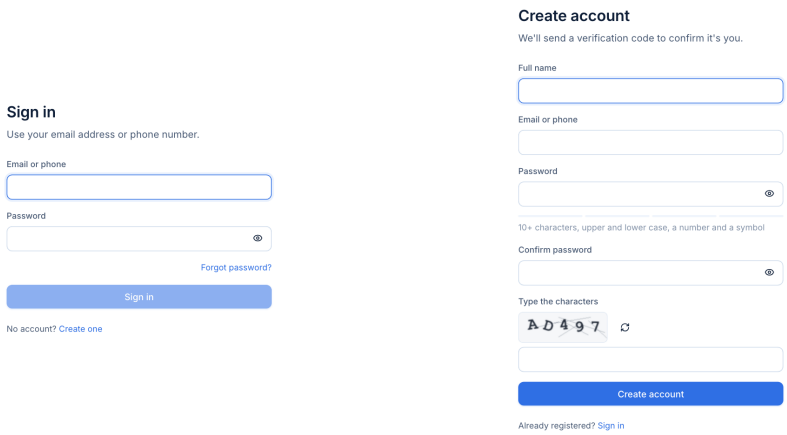

Fig 4.1: Sign-in and account creation pages

Figure 4.2 shows the dashboard. It summarises videos analysed, violations found, challans issued and cases waiting for review, explains what makes a good video, and accepts an upload with the chosen checks.

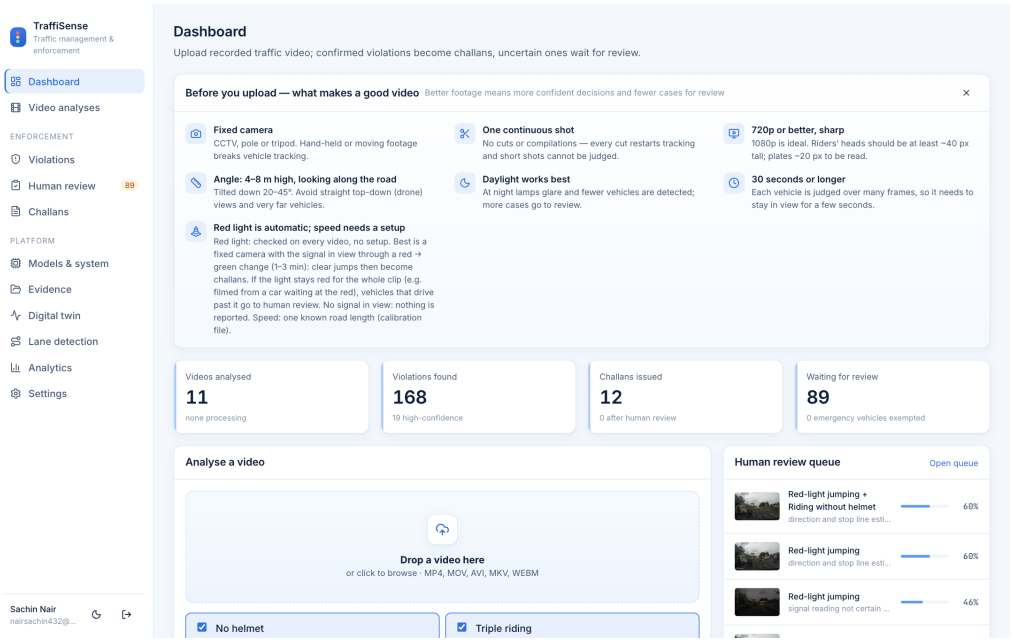

Fig 4.2: Dashboard with video guidance and upload

Figure 4.3 shows the list of video analyses with the checks run, status, findings, review cases and challans for each upload.

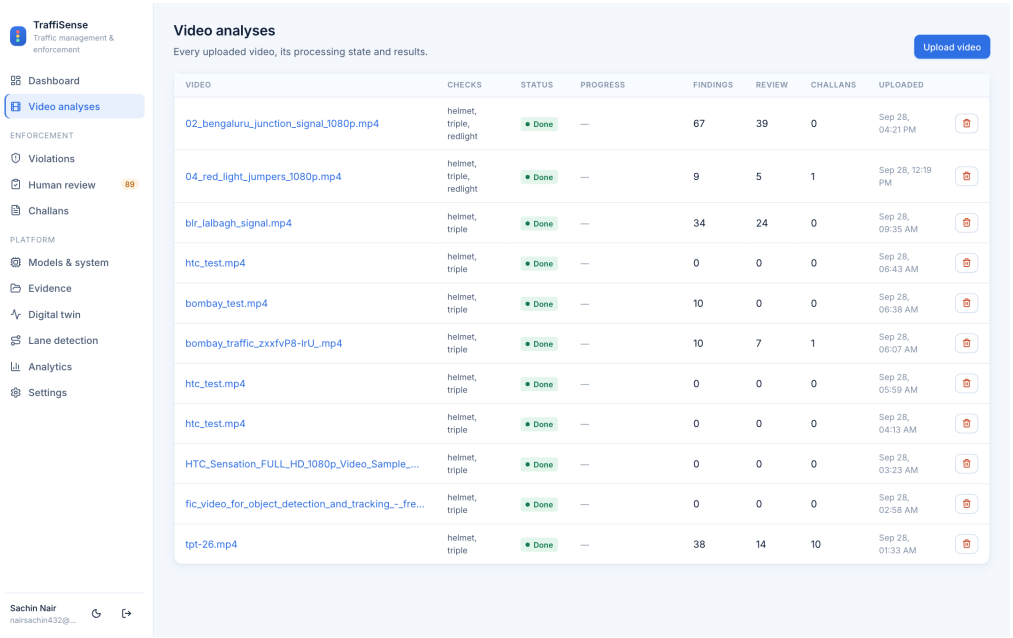

Fig 4.3: Video analyses

Figure 4.4 shows the result of one analysis. The video can be played with the helmet, triple-riding and lane overlays; violations are marked on the timeline; the results list shows each challan and review case with its confidence.

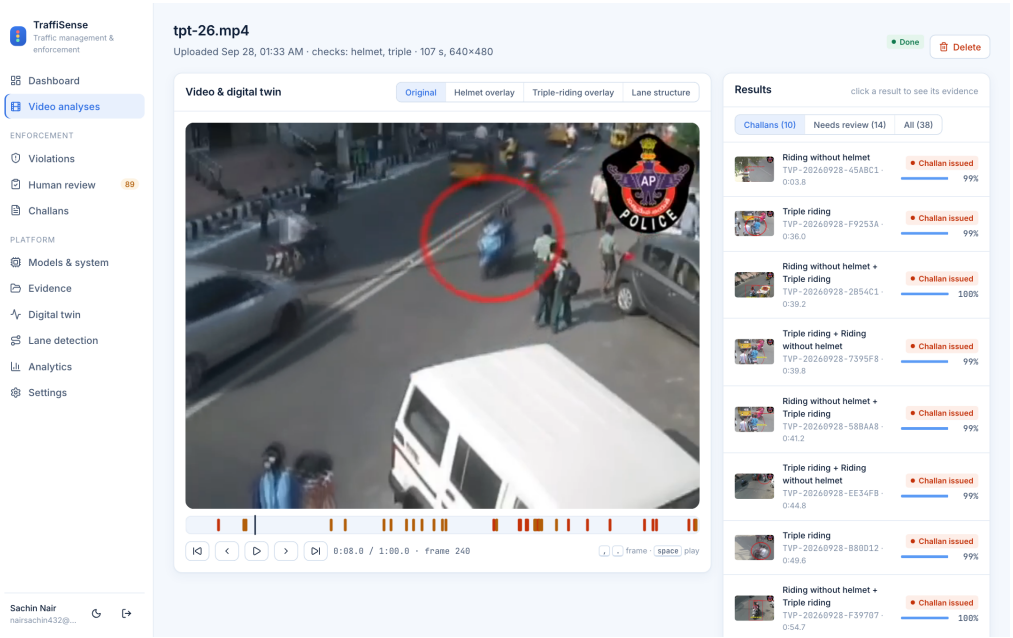

Fig 4.4: Analysis result with video, overlays and findings

Figure 4.5 shows every finding across analyses with its evidence thumbnail, track, confidence and status, including merged duplicates.

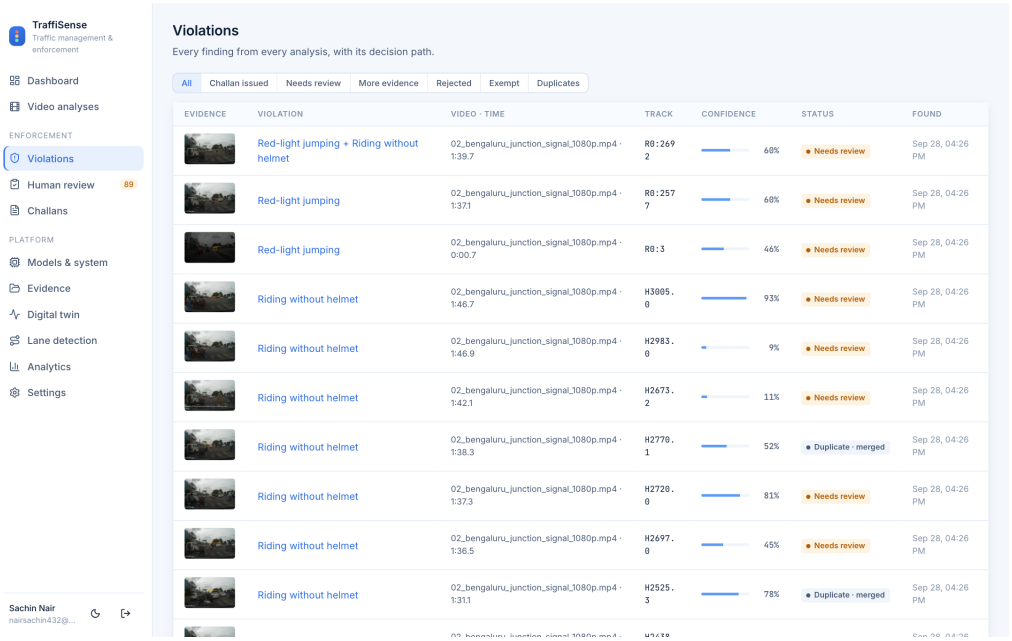

Fig 4.5: Violations with decision path

Figure 4.6 shows the human review page. The reviewer sees the evidence frames and clip, the plate field, the model's explanation and the reason it was unsure, and decides to issue a challan, reject or ask for more evidence.

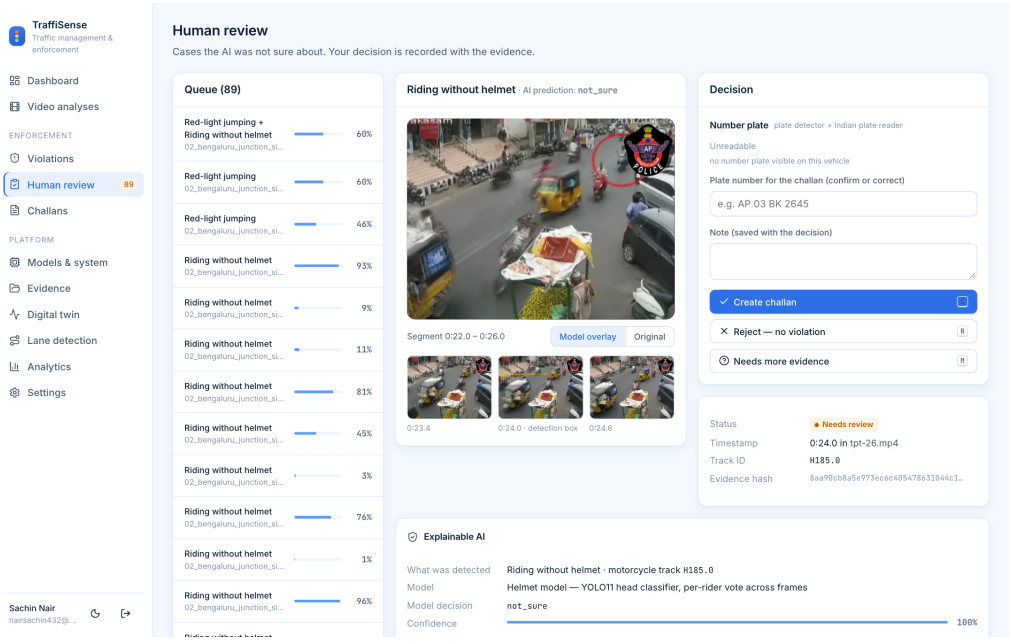

Fig 4.6: Human review of an uncertain case

Figure 4.7 shows the issued challans with their number, offence, plate, source, confidence and a PDF link.

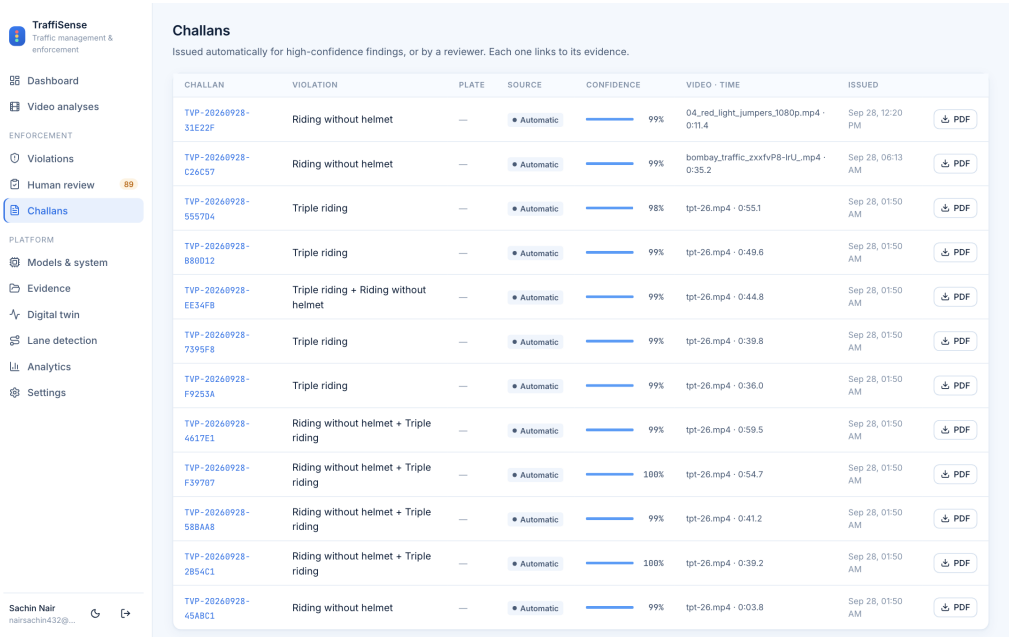

Fig 4.7: Issued challans

Figure 4.8 shows the detail of one challan with the evidence frames, the challan details and the evidence fingerprint.

Fig 4.8: Challan detail with evidence

Figure 4.9 shows the first page of an e-challan PDF generated by the platform.

Fig 4.9: First page of a generated e-challan PDF

Figure 4.10 shows the evidence gallery, where every finding keeps its frames, clips, boxes, track, confidence and fingerprint.

Figure 4.11 shows the digital twin: a 3-D reconstruction of the scene from the shared tracks, with every vehicle and violation on a timeline.

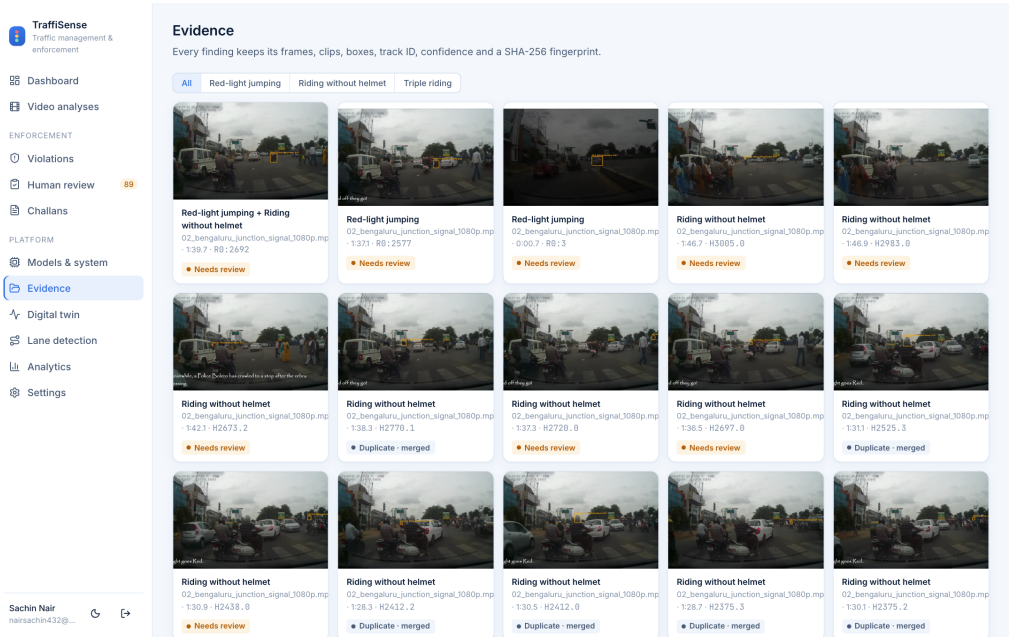

Fig 4.10: Evidence gallery

Fig 4.11: Digital twin of the scene

Figure 4.12 shows the lane detection page. Lanes are estimated from how vehicles move, not from paint, and are used as context only.

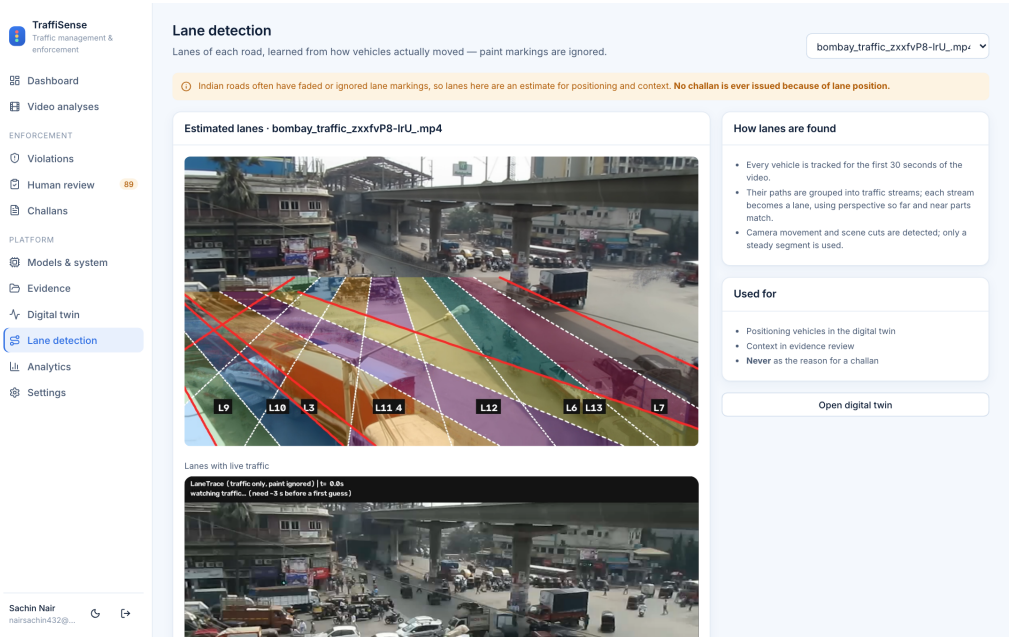

Fig 4.12: Lane detection from vehicle movement

Figure 4.13 shows the analytics page with violation types, the outcome of every finding, the confidence distribution and findings by model.

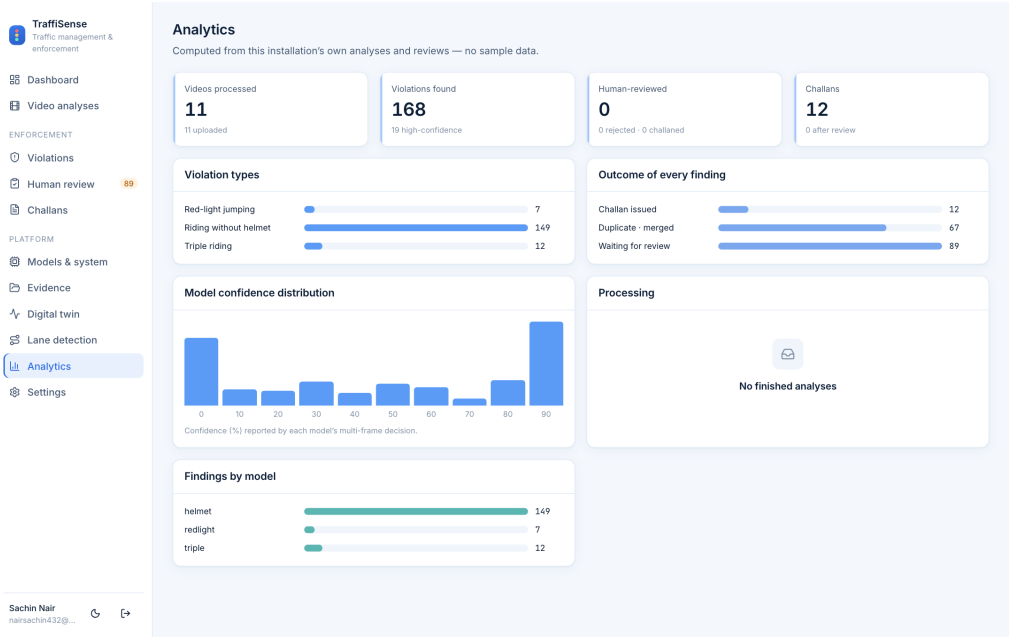

Fig 4.13: Analytics

Figure 4.14 shows the models used by the installation, their status and how decisions are made.

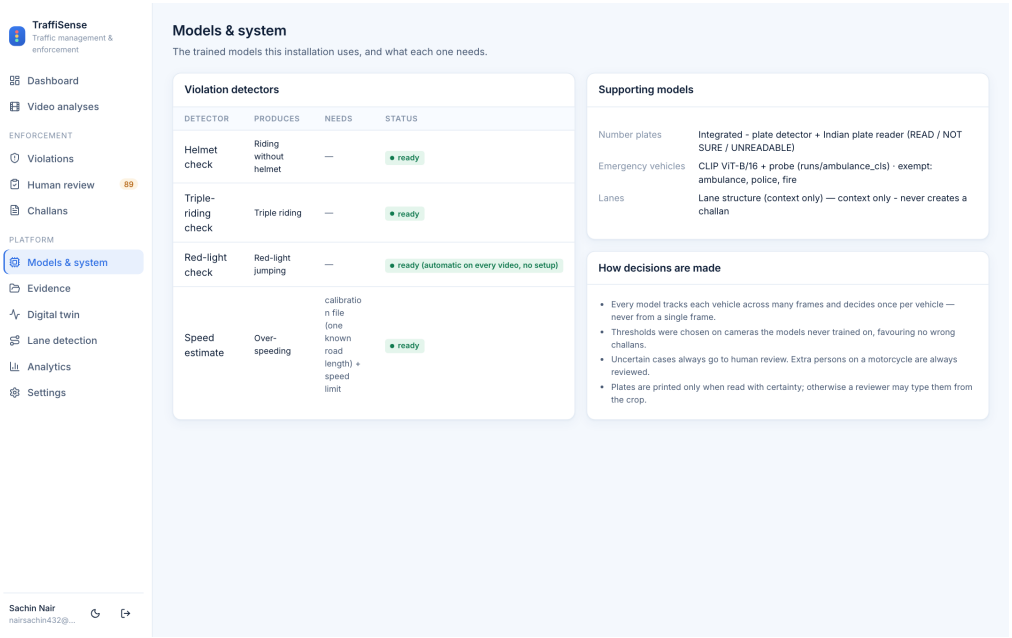

Fig 4.14: Models and system

Figure 4.15 shows the settings page for profile, password, appearance, upload defaults and code delivery.

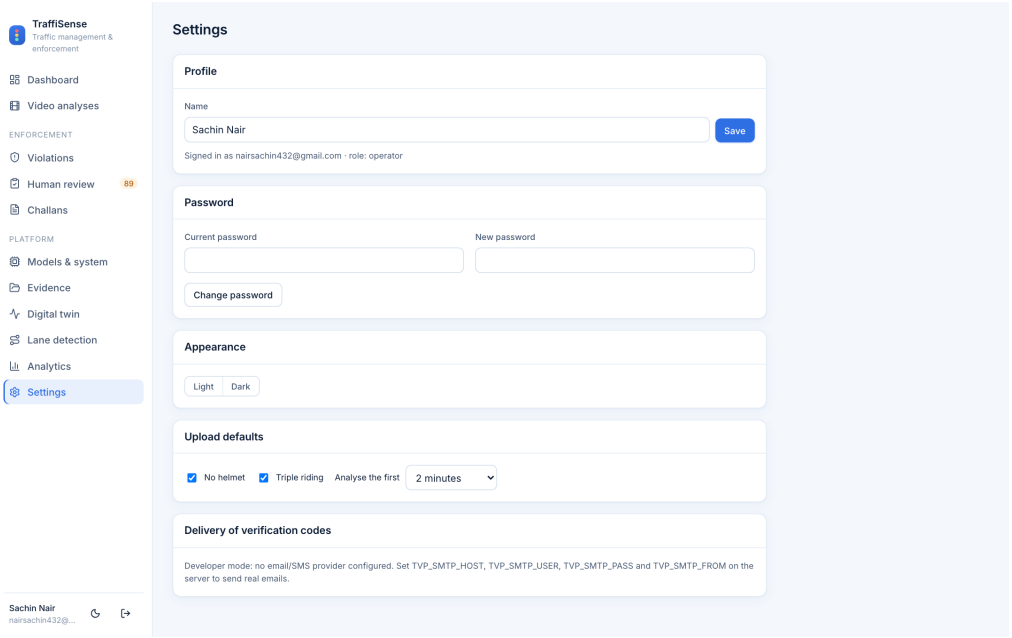

Fig 4.15: Settings

4.2 Evaluation Parameters

TraffiSense is evaluated on four groups of parameters: detection quality, decision quality, measurement quality and system behaviour. The first measures the vehicle detector, the second measures the violation checks as enforcement decisions, the third measures speed and signal readings, and the fourth measures the platform as a working application.

4.2.1 Detection Metrics

A detection is a true positive (TP) when its box overlaps a ground-truth box of the same class with IoU ≥ 0.5; unmatched detections are false positives (FP) and unmatched ground-truth boxes are false negatives (FN). Precision and recall are

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN} \tag{4.1}$$

The average precision AP of a class is the area under its precision–recall curve, and the mean average precision over C classes is

$$\text{mAP} = \frac{1}{C} \sum_{c=1}^{C} \int_0^1 P_c(R)\, dR \tag{4.2}$$

mAP@0.5 uses the IoU threshold 0.5; mAP@0.5:0.95 averages it over thresholds from 0.5 to 0.95.

4.2.2 Decision Metrics

Because TraffiSense may answer “not sure”, each check is measured as an enforcement decision. The most important number is the **wrong-challan rate**, the share of confident violation calls that are wrong; the **not-sure rate** is the share of cases sent to review; and **recall of called violations** is the share of true violations that are confirmed automatically:

$$\text{WrongRate} = \frac{FP_{call}}{TP_{call} + FP_{call}}, \qquad \text{NotSure} = \frac{N_{review}}{N}, \qquad \text{Caught} = \frac{TP_{call}}{N_{viol}} \tag{4.3}$$

For decisions with an answer, accuracy among called cases is the share of answers that are correct, and the F1-score combines precision and recall:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.4}$$

4.2.3 Measurement Metrics

For the speed estimator, the mean absolute percentage error between the middle estimate $\hat{y}_i$ and the true speed y_i, and the coverage of the 90 % range, are reported:

$$\mathrm{MAPE} = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{\hat{y}_i - y_i}{y_i}\right|, \qquad \mathrm{Coverage} = \frac{1}{n}\sum_{i=1}^{n}\mathbf{1}[y_i \in [v_{lo,i},\ v_{hi,i}]] \tag{4.5}$$

A well-calibrated range has a coverage close to 90 %. For the signal-state model, the per-class share of correctly classified lamp crops is reported. For the plate reader, the exact-plate accuracy, the character accuracy and the share of correct plates among READ decisions are reported.

For signal control in simulation, the main measure is the **mean wait per person**: the time each vehicle spends stopped before the stop line, weighted by its occupants p_v, so that a bus with forty passengers counts more than a two-wheeler with one. The wait per vehicle, the worst wait, the ambulance wait, red-light jumps per hour, throughput and the vehicles left queued at the end are reported with it:

$$\bar{W}_{person} = \frac{\sum_{v} p_v\, W_v}{\sum_{v} p_v} \tag{4.6}$$

4.2.4 System and Functional Metrics

At the system level the evaluation covers the functional correctness of each workflow, the number of findings, review cases, challans and merged duplicates per video, and the processing time compared with the length of the video (the real-time factor).

4.2.5 Evaluation Protocol

To make the reported numbers honest and reproducible, the following protocol is applied. Data are split by **camera or by video**, never by frame, because frames of the same scene are nearly identical and would leak into the test. Thresholds of the decision rules are chosen on out-of-fold predictions with the explicit target of zero (or the fewest possible) wrong calls, and the chosen operating point is then fixed. End-to-end results are additionally checked on complete videos that were not used for training or tuning, grading every reported rider or vehicle by eye. Because violations are rare, accuracy alone would look high even for a useless model, so the wrong-call count, the number caught and the not-sure rate are always reported together. Signal controllers are always compared on identical simulated arrivals (the same random seed), and differences are called better or worse only when the 95 % confidence interval of the paired difference excludes zero.

4.3 Performance Evaluation

This section reports the evaluation of each component with the parameters defined in Section 4.2, followed by platform-level results from the analyses stored in the running system.

4.3.1 Vehicle Detection

Table 4.6 reports the performance of the IDD-trained YOLO11s detector on the 4,197 held-out test images. The classes that matter for enforcement — cars, buses, trucks, auto-rickshaws, motorcycles and riders — reach mAP@0.5 between 0.66 and 0.79 with precision above 0.84. The overall mean is lowered by rare classes such as animals, caravans, trains and the generic vehicle class.

Figure 4.16 shows the validation curves during training and Figure 4.17 the per-class scores.

Table 4.6: Performance of the vehicle detector on the IDD test split

Class	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Car	0.898	0.652	0.754	0.545
Bus	0.885	0.691	0.791	0.609
Truck	0.843	0.662	0.751	0.570
Auto-rickshaw	0.881	0.688	0.784	0.583
Motorcycle	0.868	0.669	0.750	0.473
Rider	0.861	0.564	0.661	0.403
Person	0.826	0.503	0.621	0.360
Bicycle	0.739	0.483	0.543	0.329
Traffic light	0.754	0.438	0.515	0.251
All 15 classes	**0.784**	**0.480**	**0.537**	**0.355**

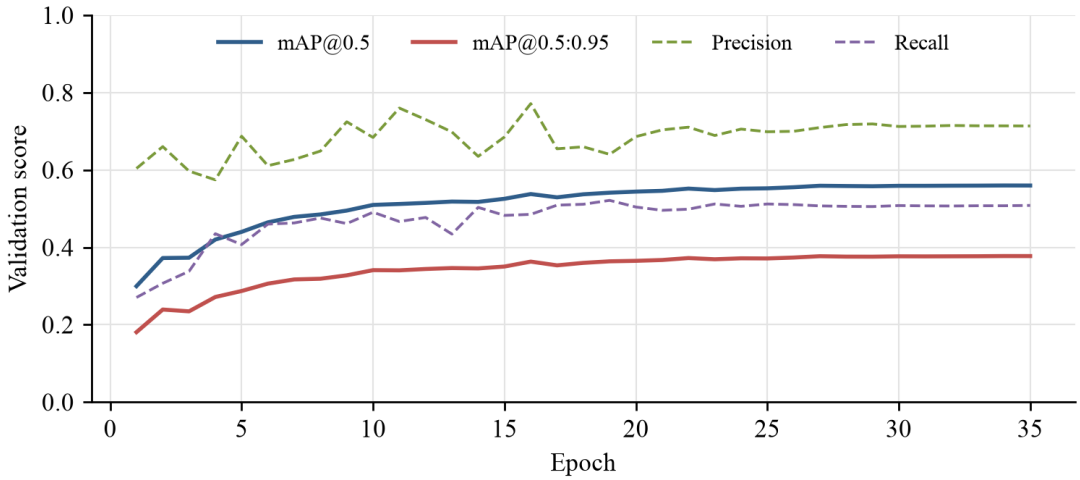

Fig 4.16: Validation precision, recall and mAP of the vehicle detector during training

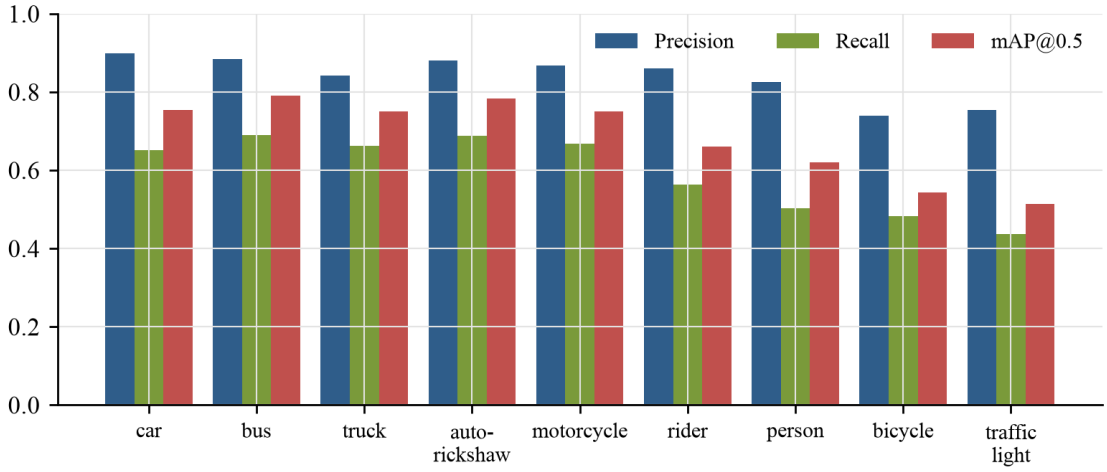

Fig 4.17: Per-class precision, recall and mAP@0.5 on the IDD test split

Recall below 0.7 means that some vehicles are missed in a single frame. Tracking compensates for much of this, because a vehicle needs to be detected only in some of the frames in which it is visible to keep its track.

4.3.2 Helmet Check

The helmet check was evaluated in two ways. Cross-validation on the 24 usable training videos (each video scored by a model that never saw it) measures the classifier; the seven unseen videos (Varanasi, Bengaluru, Hyderabad, Kochi, Mysuru, Bhopal and a Hyderabad police clip) measure the complete tool with its final precision-first rule. Table 4.7 reports both.

Figure 4.18 compares the first version of the tool with the final rule on the unseen videos. Requiring 0.98 probability over at least three agreeing frames reduced wrong no-helmet calls from 20 to 1, at the cost of more “not sure” answers, which go to review.

Table 4.7: Performance of the helmet check

Test	Measure	Result
Cross-validation, 3,366 persons (3 % not-sure setting)	Accuracy of called answers	91.8 %
	No-helmet riders caught	667 / 873 (76.4 %)
	Helmeted riders called “no helmet”	50 / 1,228
Cross-validation, final rule	Helmeted riders called “no helmet”	4 / 1,228
7 unseen videos, 367 riders, final rule	“No helmet” calls (wrong)	29 (1 wrong)
	“Helmet” calls (wrong)	175 (3 wrong)
	Wrong among checkable answers	4 / 167 (2.4 %)
	Not sure	44 % (71 of 162 unreadable even for a person)
	Clear no-helmet riders caught	27 / 81 (33 %)

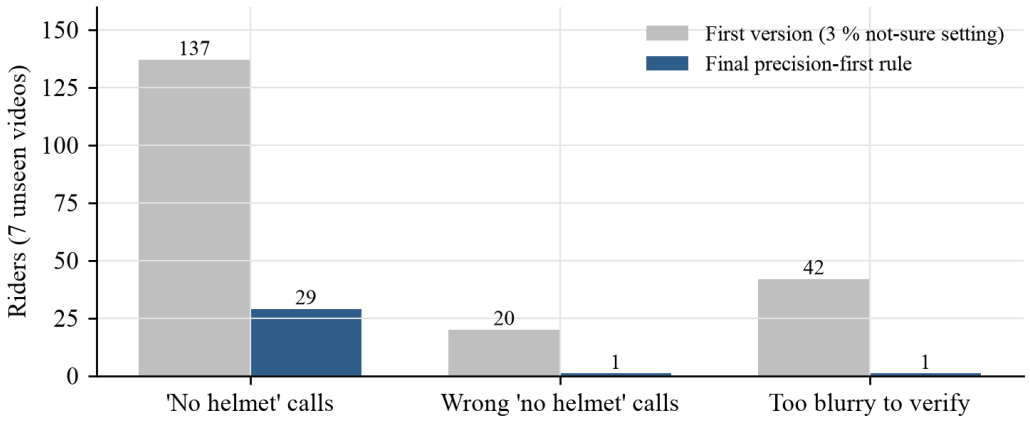

Fig 4.18: Helmet check on seven unseen videos: first version and final rule

4.3.3 Triple-Riding Check

Triple riding is rare in real footage (about one bike in 150), so it was tested on held-out cameras containing 980 two-wheelers with 125 real triples. Table 4.8 compares decision rules with the same not-sure budget of 8.3 %.

Table 4.8: Triple-riding check on held-out cameras (980 bikes, 125 triples)

Decision rule	Triples caught	Wrong TRIPLE calls	Not sure
Classifier, ≥ 1 agreeing frame	20 (16 %)	0	8.3 %
Classifier, ≥ 3 agreeing frames	37 (30 %)	0	8.3 %
Classifier + heads on seat, ≥ 3 frames (chosen)	**50 (40 %)**	**0**	**8.3 %**
Chosen rule: triples caught or sent to review	90 (72 %)	–	–

On full street-level videos from Indore, Pune and Jaipur (about 700 motorcycles), the tool made one wrong triple call (a rider with a side-saddle passenger) and missed the few visible triples, which were children seated between adults or bikes visible for less than a second. The check is therefore strong on clear police-CCTV views and weaker on busy hand-held footage.

4.3.4 Signal State and Red-Light Check

Table 4.9 reports the signal-state classifier on held-out clips and on independent long clips graded by eye.

Table 4.9: Performance of the signal-state classifier

Test	Result
Held-out LISA / TVD crops: red	2,252 / 2,252 red; TVD red 72 / 73
Held-out LISA / TVD crops: yellow, green	Yellow 72 / 72; green 1,114 / 1,132; TVD green 76 / 78
Held-out LISA crops: not a light	456 / 461
Indian crops, fine-tuned (two halves)	Real reds 25 / 26; non-lights called red 1.1–3.8 %
Fixed camera, US junction, 66 s (graded frames)	56 / 57 correct (98 %)
Night camera, Las Vegas, 5 min	38 / 41 correct when a call is made; green or yellow never read as red
Red-light crossings, Indian junction clip	2 / 2 correct (limited test material)

The colour-change safeguard means that a signboard, red car or brick paving read as a red lamp can at most cause a review, never a challan. Red-light crossings could be tested on only a few clips, because most available footage either shows no signal facing the camera or is cut into short shots; this is the main gap in the evaluation.

4.3.5 Emergency-Vehicle Exemption

Table 4.10 and Figure 4.19 show the trade-off of the emergency probe on 170 held-out emergency tracks and 2,450 ordinary tracks. At the chosen exemption threshold of 0.956, 98 emergency vehicles were exempted and no ordinary vehicle was wrongly exempted; with the review band, 143 of the 170 were exempted or reviewed.

Table 4.10: Emergency-vehicle recognition at different thresholds

Threshold	Probe: caught	Probe: wrongly flagged	Zero-shot: caught	Zero-shot: wrongly flagged
0.5	147 / 170	68 / 2,450	133 / 170	16 / 2,450
0.7	128 / 170	18 / 2,450	111 / 170	3 / 2,450
0.9	104 / 170	1 / 2,450	92 / 170	0 / 2,450
0.95	98 / 170	1 / 2,450	80 / 170	0 / 2,450
0.98	82 / 170	0 / 2,450	66 / 170	0 / 2,450

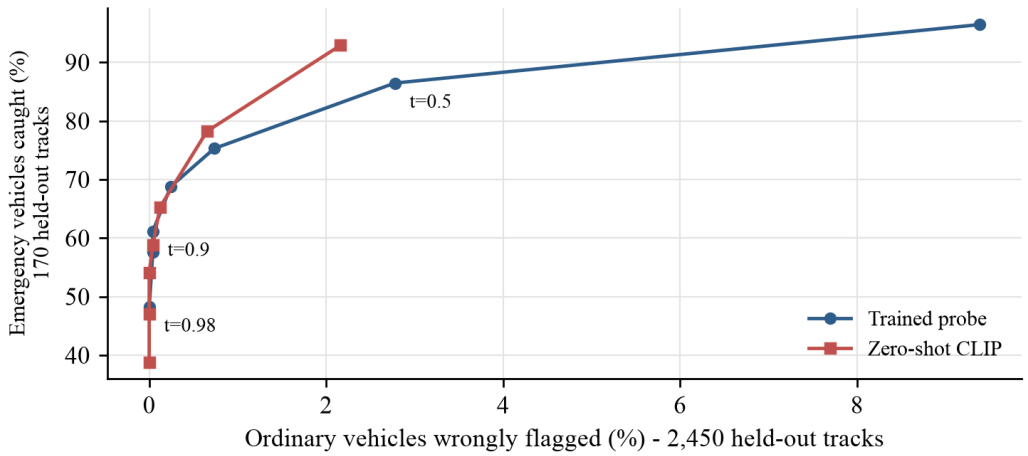

Fig 4.19: Emergency vehicles caught against ordinary vehicles wrongly flagged

4.3.6 Number-Plate Reading

Table 4.11 compares the plate reader with general OCR baselines on the 315 held-out test plates, and reports the certainty rule.

Table 4.11: Performance of the number-plate reader

Component / setting	Result
Plate detector recall (held-out plates)	99.4 %
EasyOCR (baseline), exact plate	13.7 %
Pretrained fast-plate-ocr, exact plate	45.5 %
Fine-tuned Indian reader, exact plate	81.6 %
Fine-tuned Indian reader, characters	95.4 %
Certainty rule (character confidence 0.90)	99.0 % of READ plates correct; 70 % of plates READ
Video audit of READ plates	20 / 20 correct

On real CCTV footage most plates are too small to read with certainty, so relatively few vehicles receive a READ plate; the reviewer types the plate from the crop for the others. This is the intended behaviour: no guessed plate is ever printed.

4.3.7 Speed Estimation

Table 4.12 reports the calibrated speed estimator, calibrated on one carriageway and tested on the other, so that calibration and test use different references.

Table 4.12: Calibrated speed estimation on reference scenes

Test	Vehicles	Mean error	90 % range contains truth
M6, calibrated left, tested right	33	6.3 km/h (8.4 %)	97 %
M6, calibrated right, tested left	27	4.3 km/h (11.4 %)	85 %
Autobahn, calibrated left, tested right	39	10.7 km/h (8.8 %)	100 %
Autobahn, calibrated right, tested left	38	10.3 km/h (7.6 %)	100 %
All	**137**	**≈ 9 %**	**96 %**

Without calibration every vehicle was labelled “speed ?”, so no wrong number was shown, although the hidden middle estimates were 13.6 % and 53 % off on the two scenes. Because the calibrated error is about 9 %, the platform confirms over-speeding only when the whole 90 % range is above the limit.

4.3.8 Platform-Level Results and Functional Testing

The platform was exercised with 11 uploaded videos of Indian and other traffic, including a Bengaluru junction, a Lalbagh signal, red-light jumpers, Mumbai traffic, a Tirupati police-camera clip and highway footage. Across these analyses the checks produced 168 findings: 12 automatic challans, 89 cases waiting for review and 67 repeat detections merged into existing cases. Figure 4.20 shows how the raw findings of each check were routed before merging.

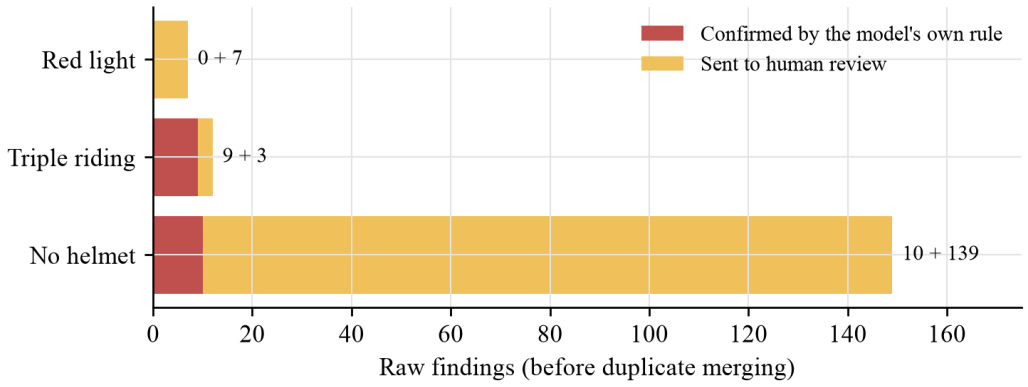

Fig 4.20: Routing of raw findings by check across all analyses

Table 4.13: Processing results of the uploaded videos

Video	Resolution	Length (s)	Processing (min)	Findings	Challans
tpt-26.mp4	640×480	107	16.9	38	10
fic_video_for_object_detect...	3840×2160	306	19.7	0	0
HTC_Sensation_FULL_HD_1080p...	1920×1080	158	7.3	0	0
htc_test.mp4	1920×1080	158	4.4	0	0
htc_test.mp4	1920×1080	158	1.7	0	0
bombay_traffic_zxxfvP8-lrU_...	1280×720	263	6.3	10	1
bombay_test.mp4	1280×720	263	2.9	10	0
htc_test.mp4	1920×1080	158	0.7	0	0
blr_lalbagh_signal.mp4	1920×1080	180	5.6	34	0
04_red_light_jumpers_1080p.mp4	1920×1080	12	0.6	9	1
02_bengaluru_junction_signa...	1920×1080	119	5.0	67	0

Figure 4.21 relates processing time to video length on the Apple M4. Most 720p–1080p videos were processed in one to two times their length; the 4K video and the long low-resolution police clip took longer, because the 4K file needs a working copy and the police clip contains many riders, each of which is classified in every frame. Repeated uploads of the same footage are recognised and their vehicles are not challaned again.

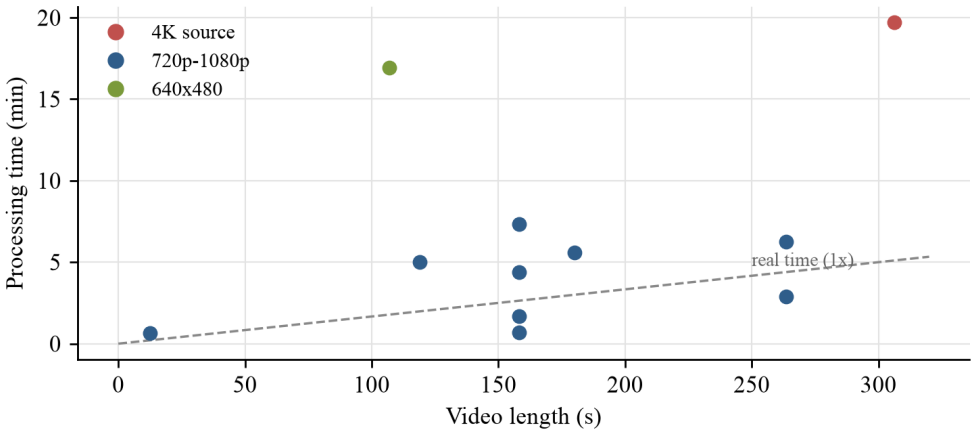

Fig 4.21: Processing time against video length

Table 4.14 lists the functional test scenarios. The status column records what was observed on the running platform; scenarios that could not yet be confirmed from stored data are marked to be recorded.

4.3.9 Adaptive Signal Control in Simulation

All results in this section are **simulated**: they come from the Unity junction simulator, whose driver-behaviour parameters (patience, dwell times) are estimates, so they show how the controllers compare rather than predict real delays. Figure 4.22 shows the simulator. Each controller was run for 30 simulated minutes in six scenarios with three random seeds, every controller receiving the same arrivals in each seed. Table 4.15 and Figure 4.23 report the mean wait per person.

Table 4.14: Functional test scenarios and observed behaviour

ID	Scenario	Expected behaviour	Status
T1	Sign up and verify	CAPTCHA and six-digit code required; account usable only after verification.	Pass (2 verified accounts)
T2	Sign in and session	HttpOnly session cookie; pages behind sign-in redirect to /login without it.	Pass
T3	Upload and background analysis	Upload returns at once; stage and progress update until done.	Pass (11 analyses done)
T4	Automatic challan	Certain findings become challans with number, offences and evidence hash.	Pass (12 issued)
T5	Challan PDF	PDF downloads with evidence, QR code and fingerprint.	Pass (HTTP 200, PDF opened)
T6	Evidence collection	Frames, crop, clips and SHA-256 stored for every finding.	Pass
T7	One case per vehicle	Repeat detections merged; at most one challan per vehicle.	Pass (67 merged)
T8	Re-upload of same footage	Vehicles already handled are not challaned again.	Pass (Mumbai re-upload: 0 new)
T9	Human review decision	Challan / reject / more evidence recorded with note and plate.	To be recorded
T10	Emergency exemption	Ambulance, police or fire vehicle exempted with reason.	To be recorded on platform
T11	Delete analysis	Merged findings elsewhere restored and re-merged.	To be recorded
T12	Rate limit and code attempts	Repeated sign-in or code attempts are blocked.	To be recorded

PRAVAH gave the lowest wait per person in every scenario, 17–47 % below the queue-pressure adaptive controller in five of them. Ambulances waited 0.1–0.4 s on average in five scenarios and 7.6 s near capacity. The gains are not uniform: near capacity PRAVAH was on par with Adaptive v1 (73.8 s against 75.7 s), with a longer worst-case wait (295 s against 268 s) and slightly more red-light jumps, and in the ambulance corridor its worst wait was 148 s against 101 s.

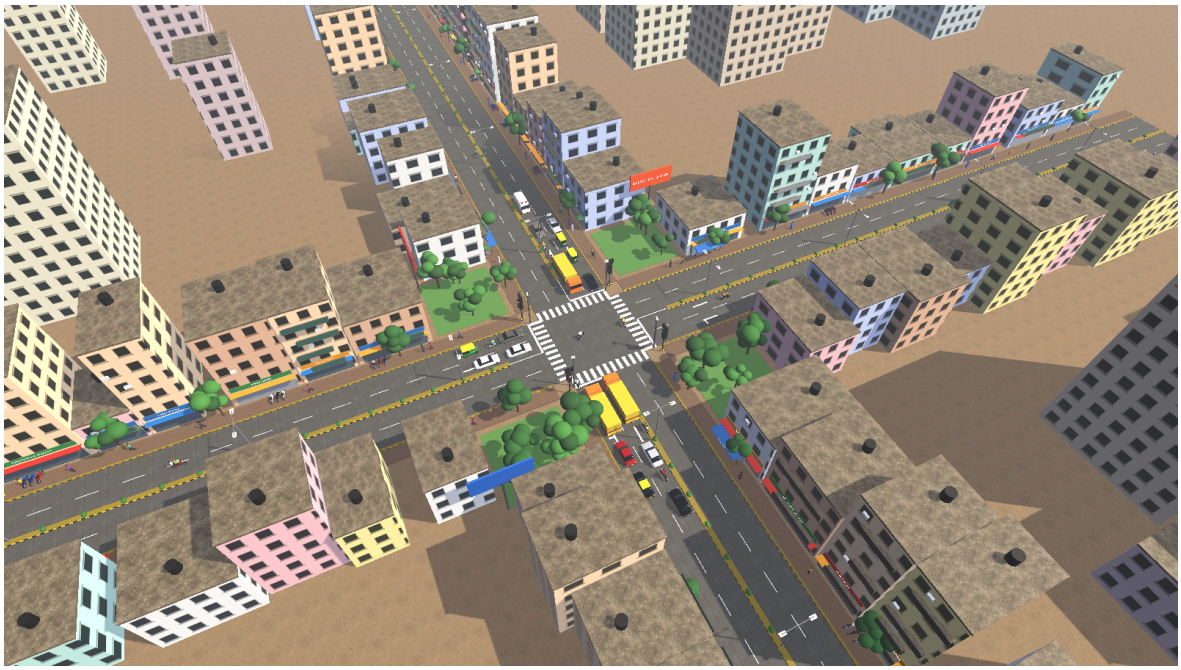

Fig 4.22: The Unity junction simulator: an Indian four-arm junction with mixed traffic

Table 4.15: Mean wait per person (s) in simulation: 30 min, 3 seeds, identical arrivals

Scenario	Fixed 30 s	Webster	Adaptive v1	PRAVAH	PRAVAH vs Adaptive v1
City balanced	38.9	36.3	26.9	**15.5**	−42 %
Office rush (N–S)	89.8	36.3	23.3	**19.4**	−17 %
Market road (cattle, bus stops)	41.5	30.7	22.3	**14.4**	−35 %
Peak near capacity	87.0	98.1	75.7	**73.8**	−3 %
Night	31.6	11.1	5.5	**2.9**	−47 %
Ambulance corridor	41.0	29.6	20.2	**15.9**	−21 %

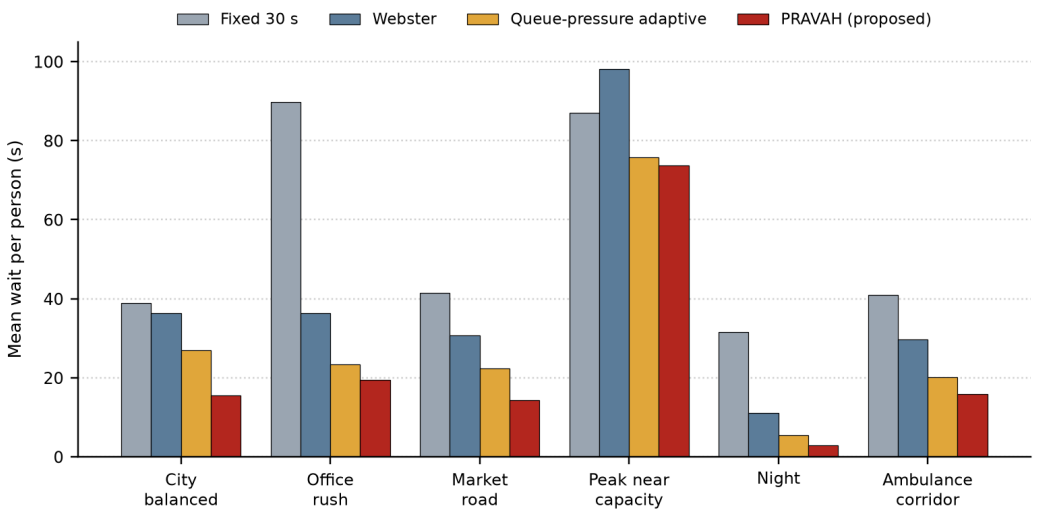

Fig 4.23: Mean wait per person by controller and scenario (lower is better)

Robustness. Because the simulator's assumptions are estimates, the comparison was repeated with each assumption changed: driver patience halved and increased by half, bus dwell halved and doubled, camera range 50 m and 150 m, demand −20 % and +20 %, a two-wheeler-heavy mix, no red-light jumping and no free left turn, with ten seeds each (2,220 runs of 30 simulated minutes). Table 4.16 shows that PRAVAH kept the lowest mean wait per person under every one of the twelve assumption sets. Paired over all 72 combinations of assumption and scenario, it was significantly better than Adaptive v1 in 60 and never significantly worse on this measure. The trade-off appears in other measures: against Adaptive v1 it had significantly more red-light jumps in 11 combinations and a longer worst wait in 4.

Python in the loop. In a shadow test, the C# and Python implementations of PRAVAH received identical junction states at every decision and disagreed in 0 of 83,702 decisions, so the Python bridge reproduces the controller exactly. Running the project's earlier rule-based signal controller through the bridge (Table 4.17) showed that it does well in moderate traffic but breaks down near capacity: the mean wait rose to 98.7 s, throughput fell to 2,339 vehicles per hour and 486 vehicles were left queued. The lack of a waiting-time limit and of a spillback check explains this, and motivated both features in PRAVAH.

Table 4.16: Robustness of the mean wait per person to the simulator assumptions

Simulator assumption	PRAVAH	Best other (Adaptive v1)	Scenarios where PRAVAH is significantly better
Baseline	23.1	27.3	5 / 6
Impatient drivers (patience × 0.5)	19.3	24.0	5 / 6
Patient drivers (patience × 1.5)	24.7	28.9	5 / 6
Short bus stops (dwell × 0.5)	18.1	23.2	6 / 6
Long bus stops (dwell × 2)	48.9	56.5	3 / 6
Short camera range (50 m)	24.0	28.3	5 / 6
Long camera range (150 m)	23.1	28.1	5 / 6
Demand −20 %	10.9	15.6	6 / 6
Demand +20 %	51.0	55.2	5 / 6
Two-wheeler-heavy mix (58 %)	15.7	20.3	6 / 6
No red-light jumping	24.9	28.8	5 / 6
No free left turn	30.6	34.7	4 / 6

Table 4.17: The project's rule-based signal controller run through the Python bridge (simulated)

Scenario	Wait per vehicle (s)	Wait per person (s)	Worst wait (s)	Throughput (veh/h)	Left queued
City balanced	29.8	26.5	220	2,373	40
Office rush (N–S)	29.4	27.3	309	2,432	33
Market road	27.6	25.1	181	2,237	35
Peak near capacity	98.7	95.0	390	2,339	486
Night	9.5	4.6	133	654	3
Ambulance corridor	23.1	20.9	169	2,195	26

4.3.10 Comparison with the Project-A Proposal

Table 4.18 compares the TraffiSense architecture proposed in Project-A with what has been implemented in this Project-B platform. The comparison is qualitative.

Table 4.18: Project-A proposal compared with the Project-B implementation

Aspect	Project-A proposal	Project-B implementation
Input	Four simulated CCTV streams of one junction	Uploaded recorded videos from any fixed camera
Detection	YOLOv8 fine-tuned on UVH-26 and IDD	YOLO11s fine-tuned on IDD, run once and shared by all checks
Violations	No helmet, triple riding, red light	No helmet, triple riding, red light, over-speeding
Multi-frame verification	K of N frames per track	Weighted votes, minimum frames, agreement and colour-change rules
ANPR	EasyOCR / PaddleOCR	Fine-tuned Indian reader with READ / NOT SURE / UNREADABLE
Emergency vehicles	Signal pre-emption	Exemption from challans with CLIP probe
Explainability	Grad-CAM heatmaps and rule reasons	Plain-language reasons, uncertainty reasons, evidence frames and clips, fingerprint
Human review	Low-confidence queue	Review page with plate entry and three decisions; one case per vehicle
Signal control and simulation	Adaptive timing validated in a SUMO digital twin	PRAVAH controller evaluated in a Unity simulator of an Indian junction (simulated results); 3-D twin of the analysed scene
Lanes	Painted lane detection for density	Lanes from vehicle movement, context only
Interface	Flask / Streamlit dashboard	React + FastAPI platform with accounts and 18 pages

4.3.11 Discussion

The implemented system shows that an evidence-first enforcement workflow can be assembled from trained components and standard web technology. Three design decisions deserve comment. First, running the vehicle detection once and sharing it gives every check, the plate reader and the duplicate merging the same track numbers, which makes it possible to say with certainty that two findings belong to the same vehicle. Second, choosing every operating point for zero (or near-zero) wrong calls on unseen cameras turns the models into tools that can be trusted when they do speak: one wrong no-helmet call in 29 on new videos, and no wrong triple-riding or emergency exemption calls on held-out cameras. Third, treating “not sure” as a first-class answer, with its reason shown to the reviewer, keeps the human in charge of exactly the cases where the machine is weak.

For signal control, counting waiting people instead of vehicles and penalising long red waits gave the largest gains where traffic is mixed and moderate, and the advantage held when the simulator's

assumptions were changed. Near capacity no controller can create capacity that the junction does not have, and PRAVAH was only on par with the adaptive baseline there. These remain simulated results; field validation with measured queues at a real junction is needed before any claim about real delays.

Overall, the results show that the two parts of TraffiSense complement each other. The enforcement part makes few wrong decisions and leaves the uncertain cases to people, and the signal-control part shows in simulation that the traffic measured by the same cameras can be used to reduce the time people wait at a junction.

The cost of this precision is recall and reviewer time. At the chosen settings the helmet check confirms only a third of clear no-helmet riders automatically and sends 44 % of riders to review on phone-camera footage, and the triple-riding check confirms 40 % of triples on police-camera views. Both numbers are much better on fixed, elevated CCTV views than on hand-held street video, which is why the dashboard guides operators towards suitable footage.

The main limitations are the following. Red-light crossings were tested on little real material. Some training and test labels were produced with AI assistance and have not been re-checked by a person. Plates on real CCTV are often too small to read with certainty. Night footage remains weaker for signals and emergency vehicles. Finally, TraffiSense is a prototype that analyses recorded video; its challans are not legal notices, and it must not be used for real enforcement without validation, certified speed measurement, legal review and institutional controls.

Chapter 5

Summary

In this report, the implementation of TraffiSense, an evidence-first traffic enforcement platform for recorded traffic video, is presented. The report began with the need for reliable and explainable enforcement of two-wheeler and signal offences on Indian roads and reviewed recent literature on adaptive signal control, emergency-vehicle priority, real-time object detection, Indian traffic datasets, two-wheeler violation detection and low-resolution number-plate recognition. The research gaps that emerged were single-frame decisions, the lack of a “not sure” outcome, guessed plates, duplicate penalties, missing emergency exemptions, weak evidence, and signal control that counts vehicles rather than people.

The implemented system answers these gaps with a React frontend of 18 pages, a FastAPI backend with an SQLite database and a model layer of trained tools. A single detection and tracking pass feeds the helmet, triple-riding, red-light and speed checks and the plate reader. Every decision is taken over many frames with thresholds chosen on unseen cameras, emergency vehicles are exempted, plates are printed only when read with certainty, repeat detections are merged so that a vehicle receives at most one challan, and every finding carries evidence frames, clips, reasons and a SHA-256 fingerprint. Uncertain cases are decided by a reviewer. For signal control, the PRAVAH controller, which weights the people waiting and penalises long red waits, gave the lowest mean wait per person of four controllers in a Unity simulator of an Indian junction, robustly across twelve sets of simulator assumptions, and the platform can replay the traffic of an analysed video through the simulator. Table 5.1 summarises what the integrated prototype delivers.

Table 5.1: Components delivered in the integrated prototype

Component	Status in the prototype
Accounts: sign-up with CAPTCHA, verification codes, sign-in, reset	Implemented
Video upload, background analysis with progress	Implemented
Shared vehicle detection and tracking (IDD YOLO11s + ByteTrack)	Implemented
Helmet check	Implemented
Triple-riding check	Implemented
Red-light check with automatic stop line	Implemented
Speed estimate	Implemented (needs a calibration and a limit)
Number-plate reader (READ / NOT SURE / UNREADABLE)	Implemented
Emergency-vehicle exemption	Implemented
Evidence, fingerprint, e-challan PDF	Implemented
Human review and one case per vehicle	Implemented
Digital twin, lane detection, analytics	Implemented (lanes as context only)
Adaptive signal control (PRAVAH) in a Unity junction simulator	Implemented (simulated results only)
Signal-timing page: video traffic replayed through the simulator	Implemented (simulated results only)
Python bridge for Python signal controllers	Implemented

The evaluation parameters, namely detection precision, recall and mAP, wrong-call rate, not-sure rate, violations caught, speed error and range coverage, and functional tests, were defined, and Chapter 4 reports them for every component and for the platform. The platform improves the consistency, transparency and traceability of enforcement decisions and focuses the reviewer's time on the cases that need a person. It remains an academic prototype whose outputs must be reviewed by an authorised human.

Future scope. The following extensions would strengthen the prototype:

- More labelled street-level and night-time data for the helmet and triple-riding checks, and a head detector that counts the people on the seat.
- Retraining the plate detector on enlarged vehicle crops from Indian street video, so that more plates are read with certainty.
- Longer fixed-camera junction footage to test red-light crossings, and re-checking the AI-assisted labels by people.
- Live camera streams instead of uploaded recordings.
- Field validation of the PRAVAH signal controller with measured queue lengths at a real junction.
- Coordination of neighbouring junctions along a corridor, and pedestrian phases in the signal plan.
- Integration with the official e-challan system, after validation with the traffic authority, certified speed measurement and the applicable legal and privacy controls.

Conclusion. TraffiSense shows that one set of junction cameras can serve both enforcement and signal control. For enforcement, multi-frame decisions, an explicit “not sure” answer, certain-only plates, emergency exemption and fingerprinted evidence make every challan explainable and keep people in charge of doubtful cases. For signal control, counting the people who wait and penalising long red waits reduced the simulated waiting time at an Indian junction. Together they form a sound basis for a responsible, field-validated deployment in the future.

References

[1] M. Tolani, G. E. Saathwik, A. Roy, L. A. Ameeth, D. B. Rao, A. Bajpai and A. Balodi, "Machine learning based adaptive traffic prediction and control using edge impulse platform," *Scientific Reports*, vol. 15, art. 17161, 2025, doi: 10.1038/s41598-025-00762-4.

[2] S. Lai, Z. Xu, W. Zhang, H. Liu and H. Xiong, "LLMLight: Large language models as traffic signal control agents," in *Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD)*, vol. 1, 2025, pp. 2335–2346, doi: 10.1145/3690624.3709379.

[3] M. Alruwaili, A. Ali, M. Almutairi, A. Alsahyan and M. Mohamed, "LSTM and ResNet18 for optimized ambulance routing and traffic signal control in emergency situations," *Scientific Reports*, vol. 15, art. 6011, 2025, doi: 10.1038/s41598-025-89651-4.

[4] Y. Tian, Q. Ye and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," arXiv:2502.12524, 2025.

[5] A. Sharma, C. Mhatre, S. Gawali, R. Bokkasam, B. Kishore, V. Pattanaik, T. Rambha, A. R. Pinjari, V. Kovvali, A. Chakraborty, P. Rathore, R. Krishnapuram and Y. Simmhan, "The Urban Vision Hackathon dataset and models: Towards image annotations and accurate vision models for Indian traffic," arXiv:2511.02563, 2025.

[6] U. U. Deshpande, G. K. O. Michael, S. D. C. S. Araujo, V. Deshpande, R. Patil, R. A. A. Chate, V. R. Tandur, S. S. Goudar, S. Ingale and V. Charantimath, "Computer-vision based automatic rider helmet violation detection and vehicle identification in Indian smart city scenarios using NVIDIA TAO toolkit and YOLOv8," *Frontiers in Artificial Intelligence*, vol. 8, art. 1582257, 2025, doi: 10.3389/frai.2025.1582257.

[7] U. U. Deshpande, S. Shanbhag, R. Patil, R. A. A. Chate, S. D. C. S. Araujo, K. Pinto, J. Arjun, M. Patil and B. Nirwani, "Automatic two-wheeler rider identification and triple-riding detection in surveillance systems using deep-learning models," *Discover Artificial Intelligence*, vol. 5, art. 104, 2025, doi: 10.1007/s44163-025-00263-3.

[8] M. Saravanan and G. K. Rajini, "Automatic detection of triples rider violations using an optimized mask R-CNN framework," *Systems Science & Control Engineering*, vol. 13, no. 1, art. 2546829, 2025, doi: 10.1080/21642583.2025.2546829.

[9] D. Rawat, K. Gupta, A. B. Roy and R. K. Sarvadevabhatla, "DashCop: Automated e-ticket generation for two-wheeler traffic violations using dashcam videos," arXiv:2503.00428, 2025.

[10] R. Laroca et al., "ICPR 2026 competition on low-resolution license plate recognition," in *Proc. Int. Conf. Pattern Recognition (ICPR)*, 2026, arXiv:2604.22506.

Acknowledgement

The development of TraffiSense: An Explainable, Digital-Twin-Driven Intelligent Traffic Management and Violation Detection System benefited immensely from the valuable guidance and continuous support provided by our project supervisor, Dr. Siddharth Hariharan. The supervisor's expertise in computer vision and proactive mentorship were instrumental in shaping the vision and technical execution of this project, and the insistence on honest evaluation and evidence-based decisions was vital in ensuring the reliability and responsible design of the system.

We are deeply grateful to Dr. Sharvari Govilkar, Head of the Computer Engineering Department, for providing our team with the necessary resources and encouragement to successfully conduct this project. This support played a critical role in enhancing both our educational experience and the development of explainable, AI-driven traffic enforcement solutions.

Our sincere thanks also go to Dr. Sandeep M. Joshi, Principal of Pillai College of Engineering, for offering us this opportunity and for fostering an academic environment that encourages advancements in responsible AI and smart-city technology.

We would also like to express our heartfelt gratitude to our mentors, faculty members and fellow peers whose valuable inputs, constructive feedback and inspiration helped refine TraffiSense.

Sachin Nair

Ranjith Iyer

Aditya Vijay Kumar

TraffiSense: An Explainable, Digital-Twin-Driven Intelligent Traffic Management and Violation Detection System

Sachin Nair
Dept. of Computer Engineering
Pillai College of Engineering
Navi Mumbai, India
nsachin23comp@student.mes.ac.in

Aditya Vijay Kumar
Dept. of Computer Engineering
Pillai College of Engineering
Navi Mumbai, India
[email]

Ranjit Iyer
Dept. of Computer Engineering
Pillai College of Engineering
Navi Mumbai, India
[email]

Dr. Siddharth Hariharan
Dept. of Computer Engineering
Pillai College of Engineering
Navi Mumbai, India
[email]

***Abstract*—Indian urban junctions combine heterogeneous, lane-less traffic with frequent helmet, triple-riding and red-light violations, while enforcement and signal timing still depend largely on manual observation and fixed plans. This paper surveys the methods that underpin automated traffic enforcement and adaptive signal control, and presents an integrated system built on them for Indian roads. Video from junction cameras is analysed with a YOLO detector trained on the India Driving Dataset and ByteTrack tracking; dedicated modules check helmet use, triple riding, red-light jumping and speed, read Indian number plates and recognise emergency vehicles for exemption. Every module is precision-first: a violation is reported only when the evidence is certain, uncertain cases are routed to a human reviewer, and confirmed cases become e-challans with fingerprinted evidence on a web platform. For signal control, a Unity-based simulator of an Indian four-arm junction models mixed traffic, red-light jumping, free left turns and bus-stop spillback, and is used to evaluate PRAVAH, a person-weighted, red-patience-aware adaptive controller proposed in this work. In simulation PRAVAH gave the lowest mean wait per person in all six test scenarios, 17–47 % below a queue-pressure adaptive controller in five of them and on par near capacity. The detection modules make few wrong calls on held-out videos at the cost of coverage, which remains the main limitation.**

Keywords*—*Intelligent Transportation Systems, Traffic Violation Detection, Adaptive Traffic Signal Control, Automatic Number Plate Recognition, Helmet Detection, Mixed Traffic, Microscopic Traffic Simulation, Computer Vision

I. INTRODUCTION

Road traffic in Indian cities is unlike the lane-disciplined traffic for which most enforcement and signal-control systems were designed. Two-wheelers make up a large share of vehicles and move between lanes, autorickshaws, buses and trucks share the same carriageway, and left turns are often free. Common violations such as riding without a helmet, carrying three people on a two-wheeler and jumping a red light are frequent, yet they are still largely detected by police officers watching camera feeds or standing at junctions. At the same time, many junctions run fixed signal plans that do not respond to the traffic actually waiting.

Computer vision has made it practical to detect vehicles in camera video in real time [4], and large datasets collected from Indian traffic cameras [5] make it possible to train detectors for local vehicle types. Recent work has applied these detectors to helmet and triple-riding violations [6]–[9]. However, an enforcement system has a stricter requirement than a detector benchmark: a wrong challan affects a citizen, so a system must know when it is not sure. Similarly, adaptive signal control has a long history [1]–[4], but most methods measure demand in vehicles and assume orderly queues, which fits Indian mixed traffic poorly.

This paper reviews ten recent works (2025–2026) on adaptive signal control, emergency-vehicle priority, object detection, Indian traffic data, two-wheeler violation detection and low-resolution number-plate recognition, identifies the gaps that matter for Indian junctions, and describes an integrated system that addresses them. The main contributions are:

1) An end-to-end, precision-first violation pipeline for Indian traffic video: shared detection and tracking, modules for helmet, triple riding, red light and speed, an Indian number-plate reader with explicit READ / NOT SURE / UNREADABLE outputs, and emergency-vehicle exemption, feeding a review and e-challan web platform.
2) A Unity-based microscopic simulator of an Indian four-arm junction with lane-less two-wheelers, red-light jumping, free left turns, bus-stop spillback and stray cattle, with a Python bridge so that controllers written in Python can run the simulated signal.
3) PRAVAH, an adaptive signal controller that weights waiting people rather than vehicles, penalises long red waits that lead to red-light jumping, learns each approach's discharge rate online, avoids feeding a blocked junction and times ambulance priority.
4) An evaluation against fixed-time, Webster-designed and queue-pressure adaptive control on identical simulated traffic, together with held-out video results for the detection modules.

II. LITERATURE SURVEY

A. Adaptive and intelligent signal control

Classical practice designs fixed-time plans with Webster's method, and adaptive systems change timing from detector measurements. Tolani et al. [1] automate signal timing from the density of approaching vehicles: proximity sensors placed upstream of the signal measure vehicle speed and density, and

a machine-learning model built on the Edge Impulse platform predicts the density and arrival time of vehicles, so that green times follow real-time demand instead of preset timers. Lai et al. [2] take a different route with LLMLight, in which a large language model receives a prompt describing current traffic and reasons about which phase to serve; a specialised backbone, LightGPT, was trained for the task, and the framework was evaluated on ten real and synthetic datasets and by fifteen human experts, improving generalisation and interpretability over reinforcement-learning controllers. Both measure demand per vehicle and assume lane-based queues.

B. Emergency-vehicle priority

Alruwaili et al. [3] address emergency response times with a system that detects ambulances from both sound and video, using an LSTM network on MFCC spectrograms for sirens and a ResNet18 network for images, and gives the ambulance priority at Raspberry Pi-controlled signals. Like most preemption schemes, priority is granted as soon as the ambulance is detected, which interrupts other approaches regardless of the queue in front of the ambulance.

C. Detection models and Indian traffic data

YOLOv12 by Tian et al. [4] brings attention mechanisms into the YOLO family while keeping real-time speed; its smallest model reaches 40.6 % mAP on COCO at 1.64 ms per image on a T4 GPU. For Indian roads, the UVH-26 dataset of Sharma et al. [5] provides 26,646 full-HD images from about 2,800 Bengaluru Safe-City CCTV cameras with 1.8 million boxes over 14 Indian vehicle classes, including two-wheelers, autorickshaws and tempo-travellers, labelled through a hackathon of 565 students, together with detection models trained on it.

D. Two-wheeler violation detection

Deshpande et al. [6] detect helmet violations in Indian smart-city scenes in two stages, a DetectNet_v2 model to localise riders followed by YOLOv8 for helmets and number plates, reporting 98.56 % helmet and 97.6 % plate detection accuracy on 1,715 images. The same group [7] detects triple riding from surveillance cameras at different heights and angles, reporting 91.42 % triple-rider, 98 % two-wheeler and 81 % plate detection accuracy. Saravanan and Rajini [8] classify double and triple riders with a modified Mask R-CNN using ResNet-50 and DenseNet-121 backbones, reaching 97.93 % triple-rider detection accuracy. Rawat et al. [9] build DashCop, an end-to-end system that generates e-tickets from dashcam video, with a module that associates riders with their motorcycles and a new annotated dataset, RideSafe-400.

E. Low-resolution number plates

Laroca et al. [10] organised the ICPR 2026 competition on low-resolution licence plate recognition, built on the LRLPR-26 dataset of 20,000 training and 3,000 test tracks, each pairing low-resolution and high-resolution images of the same plate captured by real surveillance cameras. It highlights that long capture distances and compression make plates illegible, which is exactly the situation of plates on wide junction cameras.

TABLE I
SUMMARY OF THE SURVEYED LITERATURE

SR NO	PAPER	SUMMARY
1	**M. Tolani, G. E. Saathwik, A. Roy, L. A. Ameeth, D. B. Rao, A. Bajpai and A. Balodi, "Machine Learning Based Adaptive Traffic Prediction and Control Using Edge Impulse Platform," Scientific Reports, 2025 [1]**	Proposes an autonomous signal-timing system driven by the density of vehicles approaching the signal. Proximity sensors placed at a fixed distance upstream measure vehicle speed and density, and an Edge-Impulse machine-learning model predicts density and arrival time. Signal timings are adjusted in real time instead of using preset timers, reducing congestion and delay. [1]
2	**S. Lai, Z. Xu, W. Zhang, H. Liu and H. Xiong, "LLMLight: Large Language Models as Traffic Signal Control Agents," ACM SIGKDD (KDD), 2025 [2]**	Uses a large language model as the decision agent for traffic signal control, prompted with real-time traffic conditions. A specialised backbone, LightGPT, is trained for signal control to reduce cost. Experiments on ten real and synthetic datasets and evaluation by fifteen human experts show better generalisation and interpretability than reinforcement-learning methods. [2]
3	**M. Alruwaili, A. Ali, M. Almutairi, A. Alsahyan and M. Mohamed, "LSTM and ResNet18 for Optimized Ambulance Routing and Traffic Signal Control in Emergency Situations," Scientific Reports, 2025 [3]**	Presents a real-time traffic management system to reduce emergency medical response times. Sirens are detected with an LSTM on MFCC spectrograms and ambulances in images with a transfer-learned ResNet18; the outputs drive Raspberry Pi-based signal prioritisation. Priority is granted on detection. [3]
4	**Y. Tian, Q. Ye and D. Doermann, "YOLOv12: Attention-Centric Real-Time Object Detectors," arXiv, 2025 [4]**	Introduces an attention-centric YOLO framework that keeps the speed of CNN-based detectors while gaining the accuracy of attention. YOLOv12-N reaches 40.6 % mAP with 1.64 ms latency on a T4 GPU, outperforming earlier real-time detectors. [4]
5	**A. Sharma et al., "The Urban Vision Hackathon Dataset and Models: Towards Image Annotations and Accurate Vision Models for Indian Traffic," arXiv, 2025 [5]**	Releases UVH-26: 26,646 1080p images from about 2,800 Bengaluru Safe-City CCTV cameras, annotated by 565 students with 1.8 million boxes over 14 Indian vehicle classes such as two-wheeler, auto-rickshaw and tempo-traveller, along with detection models trained on the data. [5]

SR NO	PAPER	SUMMARY
6	**U. U. Deshpande et al., "Computer-Vision Based Automatic Rider Helmet Violation Detection and Vehicle Identification in Indian Smart City Scenarios Using NVIDIA TAO Toolkit and YOLOv8," Frontiers in Artificial Intelligence, 2025 [6]**	A two-stage system: DetectNet_v2 localises riders and YOLOv8 detects helmet violations and extracts number plates. On a custom dataset of 1,715 motorcycle images from Indian roads it reports 98.56 % helmet detection and 97.6 % number-plate detection accuracy. [6]
7	**U. U. Deshpande et al., "Automatic Two-Wheeler Rider Identification and Triple-Riding Detection in Surveillance Systems Using Deep-Learning Models," Discover Artificial Intelligence, 2025 [7]**	Detects triple riding from surveillance cameras mounted at different heights and angles. A ResNet18-based DetectNet_v2 handles occlusion and distant vehicles, and YOLOv8 counts riders and reads plates, reporting 91.42 % triple-rider, 98 % two-wheeler and 81 % number-plate detection accuracy. [7]
8	**M. Saravanan and G. K. Rajini, "Automatic Detection of Triples Rider Violations Using an Optimized Mask R-CNN Framework," Systems Science & Control Engineering, 2025 [8]**	Classifies double and triple riders, with and without motorcycles, using a modified Mask R-CNN with DenseNet-121 and ResNet-50 networks. Reports 97.93 % triple-rider detection, 97.41 % double/triple classification and 98.22 % segmentation accuracy across weather conditions. [8]
9	**D. Rawat, K. Gupta, A. B. Roy and R. K. Sarvadevabhatla, "DashCop: Automated E-ticket Generation for Two-Wheeler Traffic Violations Using Dashcam Videos," arXiv, 2025 [9]**	An end-to-end system that processes vehicle-mounted dashcam video to detect triple riding and helmet non-compliance and generate e-tickets. A segmentation and cross-association module links riders to their motorcycles, and the RideSafe-400 annotated video dataset is released for evaluation. [9]
10	**R. Laroca et al., "ICPR 2026 Competition on Low-Resolution License Plate Recognition," ICPR, 2026 [10]**	Reports the first competition dedicated to low-resolution plate recognition on real surveillance data. The LRLPR-26 dataset has 20,000 training and 3,000 test tracks of paired low- and high-resolution plate images, showing how distance and compression make plates hard to read. [10]

F. Gaps for Indian junctions

Table I summarises the surveyed works. Four gaps stand out. First, the signal controllers [1], [2] measure demand in vehicles, while Indian junctions mix two-wheelers carrying one or two people with buses carrying forty, so a vehicle count is a poor measure of who is waiting; they also do not treat long red waits, which provoke red-light jumping, as a risk. Second, emergency priority [3] is granted immediately rather than timed to the queue ahead of the ambulance. Third, the violation detectors [6]–[9] are evaluated by accuracy on test images; an enforcement system must also know when it is not sure, because a wrong challan affects a citizen, and plates on wide camera views are often too small to read [10]. Fourth, enforcement and signal control are studied separately, although they use the same cameras and the same vehicle tracks.

III. PROPOSED SYSTEM

The proposed system (Fig. 1) has one shared video-analysis layer feeding two uses: enforcement, which produces evidence and e-challans, and signal control, which uses the measured traffic to choose signal timing. All modules follow one rule: an automatic decision is made only when the evidence is certain; otherwise the case goes to a human reviewer, and nothing is printed or charged on a guess.

A. Shared detection and tracking

Each video is sampled at five frames per second. A YOLO11 detector trained on the India Driving Dataset (IDD) finds vehicles, riders and people, and the ByteTrack tracker keeps one identity per vehicle across frames; track fragments of the same vehicle are merged when one starts where another ended. The same tracks are reused by every module, so a vehicle is decided once even if several modules examine it.

B. Violation modules

The helmet module classifies each rider's head on enlarged crops and votes over at least three frames, reporting "no helmet" only above a strict threshold. The triple-riding module counts riders on each two-wheeler with a "not sure" band between its two thresholds. The red-light module reads the signal state and checks stop-line crossing during red. The speed module reports a calibrated speed range with a confidence level rather than a single number. Lane structure is estimated from traffic motion for context only and never creates a challan.

C. Identity and exemption

The number-plate reader searches inside each tracked vehicle, enlarged to 640 pixels, reads each plate crop with a recogniser fine-tuned on Indian plates, votes character by character across frames and checks the Indian registration format. Each vehicle is labelled READ, NOT SURE or UNREADABLE, and only READ plates are printed. An emergency-vehicle classifier on CLIP image features exempts ambulances, police and fire vehicles, sending borderline cases to review.

D. Review and e-challan platform

A FastAPI and React web platform stores each analysis, shows the video with every finding on a timeline, routes uncertain findings to reviewers and issues a PDF e-challan for confirmed cases. Every challan carries fingerprinted evidence frames and a plain-language explanation of why it was raised.

E. Signal control and simulation

The signal-control part is evaluated in a simulator of an Indian four-arm junction, described in Section IV. The measured traffic of an analysed video can be loaded into the simulator, so that controllers can be compared on traffic

recorded at a real junction; such results are always labelled as simulated.

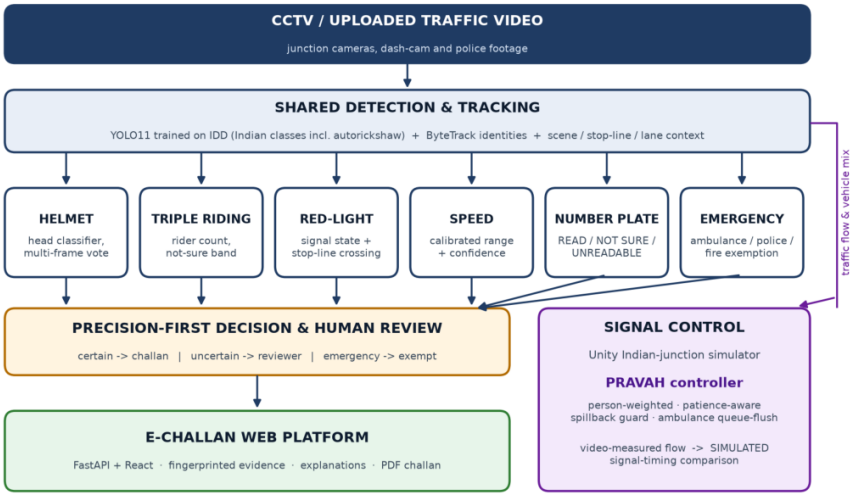

Fig. 1. Architecture of the proposed system: shared video analysis feeding precision-first enforcement and signal control.

F. PRAVAH controller

PRAVAH (Person-weighted, Red-patience-Aware, Vehicle-mix Adaptive Headway control) serves one approach at a time, which is common at Indian junctions. It uses only what the junction camera reports within 100 m of the stop line. A person who has waited w seconds contributes an urgency

$$U(w) = 1 + (w / \tau)^2, \ \tau = 75 \text{ s} \quad (1)$$

so that long waits, which lead to red-light jumping, become expensive quickly. Each waiting vehicle counts its typical occupants (about 40 for a bus, 1.4 for a two-wheeler). The time needed to clear approach a is

$$C_a = 2 + Q_a / \hat{s}_a \quad (2)$$

where Q_a is the queued load in passenger-car units and $\hat{s}_a$ the discharge rate, initialised from the Indian Roads Congress saturation flow of 525 PCU/h per metre of approach width and learned online from the vehicles that actually cross during green. The gain of serving approach a next is

$$G_a = \Sigma\, p_i\, U(w_i) \cdot \varphi(o_a) / (C_a + L) \quad (3)$$

where p_i are occupants, L = 5 s is the amber and all-red loss, and $\varphi(o) = \text{clip}(1 - (o - 0.5)/0.4, 0, 1)$ discounts an approach whose exit is more than half full, so the controller does not feed a blocked junction box. After clearing its planned queue, a green continues only while it relieves more urgency per second ($\hat{s}_a$ times the urgency per PCU of the remaining queue) than the best other approach would; any approach that has waited 90 s is served next. For an ambulance, the switch is timed so that its queue has cleared when it arrives: the green changes when its estimated arrival time falls below its clearance time plus L and 2 s.

IV. IMPLEMENTATION

The video pipeline is written in Python with Ultralytics YOLO11, ByteTrack, OpenCV and PyTorch; detectors and classifiers were trained on Kaggle T4 GPUs. The number-plate recogniser is a fine-tuned compact transformer OCR model exported to ONNX. The web platform uses FastAPI with SQLite and a React and TypeScript front end.

The simulator is built in Unity 6 with C#. Its core is a headless microscopic model with a 0.1 s step: vehicles follow the Intelligent Driver Model, drive on the left, and arrive with a typical Indian mix (38 % two-wheelers, 28 % cars, 16 % autorickshaws, 8 % trucks, 6 % buses, 4 % bicycles), weighted with Indian Roads Congress passenger-car units (0.5 for a two-wheeler, 1.2 for an autorickshaw, 2.2 for a bus or truck). Two-wheelers take lateral positions across the lane and creep past the stop line; each driver has a patience after which they jump the red; left turns are free; buses and autos stop just past the junction; a vehicle stuck in the junction box blocks cross traffic; stray cattle occasionally block the kerb lane. A three-dimensional view with day, night and monsoon lighting and eight camera modes, including junction CCTV views with detection boxes, is used for demonstration (Fig. 2).

Four controllers are compared on identical arrivals: a fixed 30 s rotation, a fixed-time plan designed with Webster's method [1] from the true flows, a queue-pressure adaptive controller, and PRAVAH. A TCP bridge lets Python controllers run the simulated signal; a shadow test, in which the C# and Python implementations of PRAVAH received identical states, found no disagreement in 83,702 decisions. The web platform can run all controllers on the traffic measured from an analysed video.

Fig. 2. The Unity junction simulator: chase view behind an ambulance on an Indian arterial (CC0 photographic sky and road textures).

V. RESULTS AND DISCUSSION

A. Signal control in simulation

Each controller was run for 30 simulated minutes in six scenarios with three random seeds, all controllers receiving the same arrivals in each seed. Table II and Fig. 3 report the mean wait per person, that is, time stopped before the stop line weighted by occupants. PRAVAH gave the lowest wait per person in every scenario. Compared with the queue-pressure adaptive controller it was 42 % lower in balanced city traffic, 17 % lower in the office rush, 35 % lower on the market road, 47 % lower at night and 21 % lower in the ambulance-heavy scenario. Ambulances waited 0.1–0.4 s on average in five scenarios and 7.6 s near capacity.

The gains are not uniform. Near capacity PRAVAH was on par with the adaptive controller (73.8 s against 75.7 s), with a longer worst-case wait (295 s against 268 s) and slightly more red-light jumps. In the ambulance-heavy scenario its worst wait was 148 s against 101 s. These results come from a simulator whose driver-behaviour parameters (patience, dwell times) are estimates, so they show how the controllers compare rather than predict real delays.

Running an earlier, rule-based adaptive controller of the project through the Python bridge gave a further finding: it performed well in moderate traffic but broke down near capacity, with a mean wait of 98.7 s, throughput falling to 2,339 vehicles/h and 486 vehicles left queued. The lack of a waiting-time limit and of a spillback check explains this, and motivated both features in PRAVAH.

TABLE II
MEAN WAIT PER PERSON (S) IN SIMULATION — 30 MIN, 3 SEEDS, IDENTICAL ARRIVALS

Scenario	Fixed 30 s	Webster	Queue-pressure adaptive	PRAVAH (proposed)	PRAVAH vs adaptive
City balanced	38.9	36.3	26.9	**15.5**	−42 %
Office rush (N–S)	89.8	36.3	23.3	**19.4**	−17 %
Market road (cattle, bus stops)	41.5	30.7	22.3	**14.4**	−35 %
Peak near capacity	87.0	98.1	75.7	**73.8**	−3 %
Night	31.6	11.1	5.5	**2.9**	−47 %
Ambulance corridor	41.0	29.6	20.2	**15.9**	−21 %

TABLE III
DETECTION MODULES ON HELD-OUT DATA

Module	Held-out test	Result	Design choice
Number-plate reader	299 Indian plate images	81.6 % plates and 95.6 % characters exact; at the operating point it answers for 64 %, 99.5 % correct	Reads only when certain
Helmet	7 unseen videos, 367 riders	4 wrong decisions; 44 % sent to review; 27 of 81 no-helmet riders caught	Avoid wrong challans
Triple riding	Held-out cameras, 980 bikes, 125 triples	50 triples caught (40 %), 0 wrong, 8.3 % not sure	Never wrong on test
Emergency exemption	170 emergency, 2,450 ordinary vehicles	58 % exempted automatically, 84 % exempted or reviewed; 0 ordinary vehicles wrongly exempted	No wrongful exemption
Speed	Calibrated demonstration video	Mean error 9.1 %; 90 % range contains the true speed for 90 % of vehicles	Range, not a point value

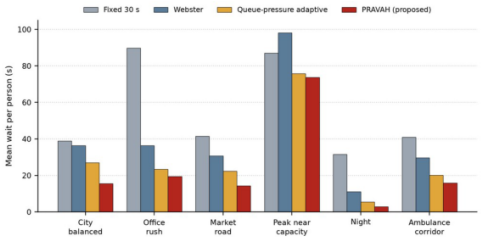

Fig. 3. Mean wait per person by controller and scenario (lower is better).

B. Detection modules on held-out data

Table III summarises the detection modules on data not used for training. All operating points were chosen to avoid wrong decisions, so coverage is the limiting factor. On still plate images the reader answers for 64 % of plates and is correct for 99.5 % of those; on street video, however, plates are often too small, and only about 1–3 % of vehicles received a READ result in the test clips. Retraining the plate detector on enlarged vehicle crops from Indian street video is in progress to address this.

VI. FUTURE SCOPE

The most important next step is coverage of the detection modules on street-level video, where plates and riders are small: retraining the plate detector on enlarged vehicle crops, and labelling more camera views, including phone footage, for helmet and triple-riding detection. For signal control, a sensitivity study over driver patience, dwell times, camera range and demand is under way to test whether PRAVAH's advantage holds when the simulator's assumptions change. Field validation with measured queue lengths at a real junction, coordination of neighbouring junctions along a corridor, pedestrian phases and deployment on edge devices beside the camera are further directions.

VII. CONCLUSION

This paper reviewed ten recent works (2025–2026) on adaptive signal control, emergency priority, detection, Indian traffic data, two-wheeler violations and low-resolution number plates, and identified four gaps for Indian junctions: demand measured in vehicles rather than people and long waits not treated as a cause of red-light jumping, emergency priority not timed to the queue, detection evaluated by accuracy rather than by its ability to abstain, and enforcement separated from signal control. The proposed system addresses them with a precision-first violation pipeline and e-challan platform, an Indian junction simulator, and the PRAVAH controller. In simulation PRAVAH reduced the mean wait per person in every scenario tested, by 17–47 % against a queue-pressure adaptive controller in five of six scenarios, while being on par near capacity. On held-out video the detection modules made few wrong decisions, and their limited coverage on street-level footage is the main area for further work.

REFERENCES

[1] M. Tolani, G. E. Saathwik, A. Roy, L. A. Ameeth, D. B. Rao, A. Bajpai and A. Balodi, "Machine learning based adaptive traffic prediction and control using edge impulse platform," Scientific Reports, vol. 15, art. 17161, 2025, doi: 10.1038/s41598-025-00762-4.

[2] S. Lai, Z. Xu, W. Zhang, H. Liu and H. Xiong, "LLMLight: Large language models as traffic signal control agents," in Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD), vol. 1, 2025, pp. 2335–2346, doi: 10.1145/3690624.3709379.

[3] M. Alruwaili, A. Ali, M. Almutairi, A. Alsahyan and M. Mohamed, "LSTM and ResNet18 for optimized ambulance routing and traffic signal control in emergency situations," Scientific Reports, vol. 15, art. 6011, 2025, doi: 10.1038/s41598-025-89651-4.

[4] Y. Tian, Q. Ye and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," arXiv:2502.12524, 2025.

[5] A. Sharma, C. Mhatre, S. Gawali, R. Bokkasam, B. Kishore, V. Pattanaik, T. Rambha, A. R. Pinjari, V. Kovvali, A. Chakraborty, P. Rathore, R. Krishnapuram and Y. Simmhan, "The Urban Vision Hackathon dataset and models: Towards image annotations and accurate vision models for Indian traffic," arXiv:2511.02563, 2025.

[6] U. U. Deshpande, G. K. O. Michael, S. D. C. S. Araujo, V. Deshpande, R. Patil, R. A. A. Chate, V. R. Tandur, S. S. Goudar, S. Ingale and V. Charantimath, "Computer-vision based automatic rider helmet violation detection and vehicle identification in Indian smart city scenarios using NVIDIA TAO toolkit and YOLOv8," Frontiers in Artificial Intelligence, vol. 8, art. 1582257, 2025, doi: 10.3389/frai.2025.1582257.

[7] U. U. Deshpande, S. Shanbhag, R. Patil, R. A. A. Chate, S. D. C. S. Araujo, K. Pinto, J. Arjun, M. Patil and B. Nirwani, "Automatic two-wheeler rider identification and triple-riding detection in surveillance systems using deep-learning models," Discover Artificial Intelligence, vol. 5, art. 104, 2025, doi: 10.1007/s44163-025-00263-3.

[8] M. Saravanan and G. K. Rajini, "Automatic detection of triples rider violations using an optimized mask R-CNN framework," Systems Science & Control Engineering, vol. 13, no. 1, art. 2546829, 2025, doi: 10.1080/21642583.2025.2546829.

[9] D. Rawat, K. Gupta, A. B. Roy and R. K. Sarvadevabhatla, "DashCop: Automated e-ticket generation for two-wheeler traffic violations using dashcam videos," arXiv:2503.00428, 2025.

[10] R. Laroca et al., "ICPR 2026 competition on low-resolution license plate recognition," in Proc. Int. Conf. Pattern Recognition (ICPR), 2026, arXiv:2604.22506.